

A Comprehensive Survey on Retrieval Methods in Recommender Systems

JUNJIE HUANG, Shanghai Jiao Tong University, China

JIZHENG CHEN, Shanghai Jiao Tong University, China

JIANGHAO LIN, Shanghai Jiao Tong University, China

JIARUI QIN, Shanghai Jiao Tong University, China

ZIMING FENG[†], China Merchants Bank Credit Card Center, China

WEINAN ZHANG[†], Shanghai Jiao Tong University, China

YONG YU, Shanghai Jiao Tong University, China

In an era dominated by information overload, effective recommender systems are essential for managing the deluge of data across digital platforms. Multi-stage cascade ranking systems are widely used in the industry, with retrieval and ranking being two typical stages. Retrieval methods sift through vast candidates to filter out irrelevant items, while ranking methods prioritize these candidates to present the most relevant items to users. Unlike studies focusing on the ranking stage, this survey explores the critical yet often overlooked retrieval stage of recommender systems. To achieve precise and efficient personalized retrieval, we summarize existing work in three key areas: improving similarity computation between user and item, enhancing indexing mechanisms for efficient retrieval, and optimizing training methods of retrieval. We also provide a comprehensive set of benchmarking experiments on three public datasets. Furthermore, we highlight current industrial applications through a case study on retrieval practices at a specific company, covering the entire retrieval process and online serving, along with practical implications and challenges. By detailing the retrieval stage, which is fundamental for effective recommendation, this survey aims to bridge the existing knowledge gap and serve as a cornerstone for researchers interested in optimizing this critical component of cascade recommender systems.

CCS Concepts: • **Information systems** → **Recommender systems**.

Additional Key Words and Phrases: Recommender System, Retrieval Stage, Candidate Generation

ACM Reference Format:

Junjie Huang, Jizheng Chen, Jianghao Lin, Jiarui Qin, Ziming Feng[†], Weinan Zhang[†], and Yong Yu. 2024. A Comprehensive Survey on Retrieval Methods in Recommender Systems. 1, 1 (March 2024), 41 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

We are being bombarded with a vast amount of information due to the growing popularity of the Internet and the development of User Generated Content (UGC) [85] in recent years. To save users from information overload, recommender systems have been widely applied in today's short video [107], news [158] and e-commerce [36] platforms.

[†]Corresponding authors.

Authors' Contact Information: Junjie Huang, Shanghai Jiao Tong University, China, huangjunjie2019@sjtu.edu.cn; Jizheng Chen, Shanghai Jiao Tong University, China, humihuadechengzhi@sjtu.edu.cn; Jianghao Lin, Shanghai Jiao Tong University, China, chiangel@sjtu.edu.cn; Jiarui Qin, Shanghai Jiao Tong University, China, qjr1996@sjtu.edu.cn; Ziming Feng[†], China Merchants Bank Credit Card Center, China, zimingfzm@cmbchina.com; Weinan Zhang[†], Shanghai Jiao Tong University, China, wenzhang@sjtu.edu.cn; Yong Yu, Shanghai Jiao Tong University, China, yyu@sjtu.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2024 ACM.

Manuscript submitted to ACM

While complicated models [102, 127, 128, 156] often offer higher accuracy, their poor efficiency makes online deployment challenging because of latency restrictions [126]. On the other hand, simple models [72, 131] have capacity limitations, but they could evaluate a great number of items efficiently because of their low time complexity. Therefore, striking a balance between efficacy and efficiency becomes crucial in order to quickly filter out information that users are interested in. As is shown in Figure 1 (a), one widely used solution in the industry is multi-stage cascade ranking systems [162]. The system includes a retriever and a variety of subsequent rankers. In the very first stage of the cascade system, referred to as the retrieval stage in this paper (also called matching stage or recall stage in some literature [129, 215]), a retriever is typically used to quickly eliminate irrelevant items from a large pool of candidates, whereas rankers in the later stages aim to accurately rank the items. Each stage selects the top- K items it receives and feeds them to the next stage. As shown in Figure 1 (a), rankers in multi-stage cascade ranking systems are arranged in the shape of a funnel, narrowing from bottom to top. The retrieval and ranking stage are two typical stages, while pre-ranking [171] and re-ranking [175] stages are relatively optional, and the number of rankers in the system may vary depending on different scenarios. Additionally, on the left side of Figure 1 (a), we display the approximate output scale of each stage, noting that the range of this scale is specific to the particular platform and scenario.

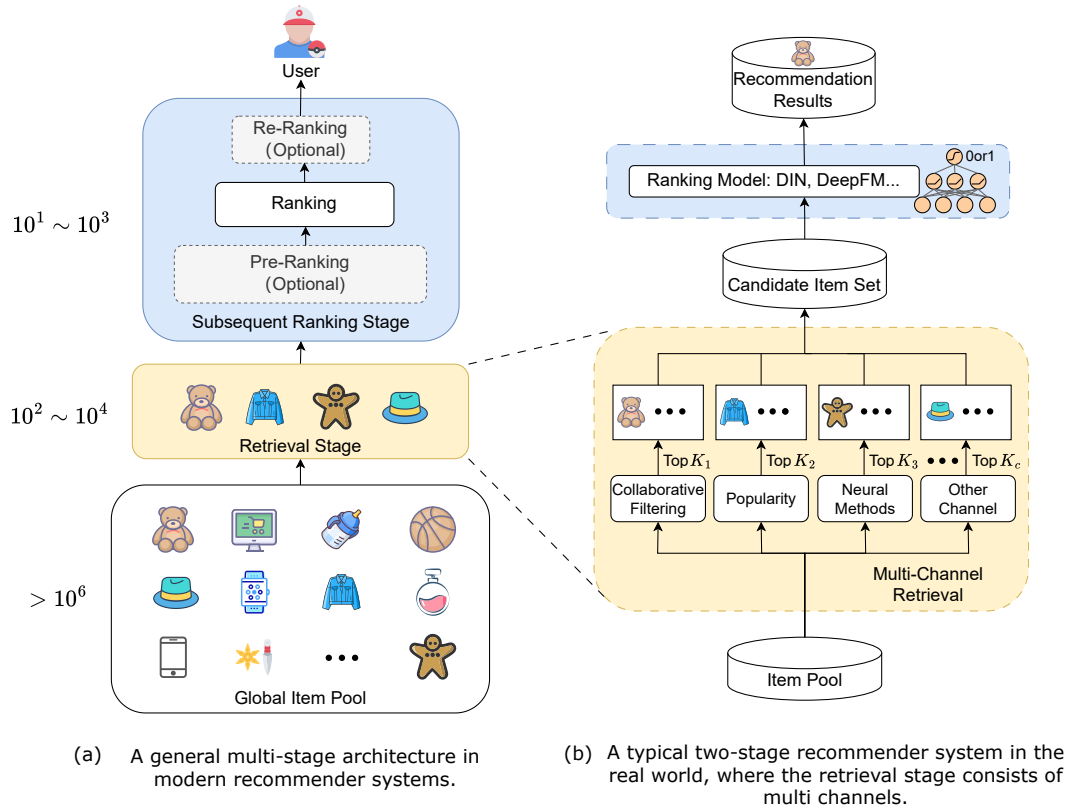


Fig. 1. The multi-stage architecture in modern recommender systems and the illustration of multi-channel retrieval. The latter will be detailed further in Section 2.4.

Although both the retrieval and ranking stages aim to select the most relevant items, each stage has its own unique characteristics.

- **Difference in candidate sets (i.e., inference spaces).** The retrieval stage needs to quickly filter through the entire item pool, which may contain millions of items; while the ranking stage only needs to score and order the items that have been selected by the retrieval methods, typically narrowing down to hundreds or thousands of items.
- **Difference in input features.** During the retrieval stage, due to time constraints and the need to filter through a large candidate set quickly, utilizing complex feature interactions is impractical for real-time online requirements. As a result, only limited, coarse-grained features of users and items are considered. In contrast, the ranking stage can utilize a diverse set of features by designing various feature interaction operators, such as product operators [130], convolutional operators [96], and attention operators [177]. The ranking stage further enhances its capability by integrating multiple feature domains and providing detailed modeling of behavioral and interaction features. These intricate and meaningful patterns serve as valuable indicators for evaluating user preferences, allowing for a more precise and refined ranking of the items.
- **Difference in model architectures.** Typically, retrieval models employ a dual-tower architecture, where user and item representations are generated separately, and their matching scores are calculated using basic similarity metrics [72], such as inner product or cosine similarity. This dual-tower architecture allows for independent optimization of user and item representations before they interact, which is often referred to as *late interaction*. On the other hand, ranking models often utilize a single-tower architecture, where user features, item features, and cross features are concatenated and processed through a deep neural network (DNN). This setup, known as *early interaction*, facilitates multi-level interactions between features, thereby increasing the model's expressive power. The single-tower architecture processes these concatenated features deeply within the network, allowing for richer and more complex interactions compared to the more straightforward combination in dual-tower setups. The distinction between single and dual-tower architectures essentially lies in how and where the interactions between users and items are modeled, that is, late interaction in dual-tower setups via direct similarity metrics, versus early interaction in single-tower setups through deeper, concatenated feature processing.
- **Difference in optimization objectives.** The retrieval stage aims to achieve high recall efficiently, i.e., to quickly identify a subset containing as many relevant items as possible. In this stage, the focus is on ensuring that the relevant items are included in the set, without prioritizing their exact order. In contrast, the goal of the ranking stage is to refine the retrieved candidate set by accurately determining the relative importance of each item. Since only a small number of items are fed into the ranking model, these models can utilize more complex architectures to enhance precision, ensuring that the most relevant items are positioned at the top of the list. Thus, the retrieval stage prioritizes the inclusion of relevant items (i.e., recall), while the ranking stage prioritizes precise ordering to maximize the relevance of the top-ranked items.

In the past few decades, there has been substantial progress in the retrieval stage. The traditional retrieval methods are based on the concept of collaborative filtering (CF) [142], which include neighborhood based [137] and matrix factorization (MF) based methods [84, 115]. However, these methods primarily focus on learning embeddings for user and item IDs, without considering other important features such as user profiles, item attributes, or contextual information. Additionally, using just shallow structures in these models is insufficient to capture complex patterns and interactions, resulting in suboptimal representations. Consequently, in recent years, there has been a shift towards embedding based retrieval (EBR), which treats the retrieval problem as a nearest neighbor search in the vector space. EBR

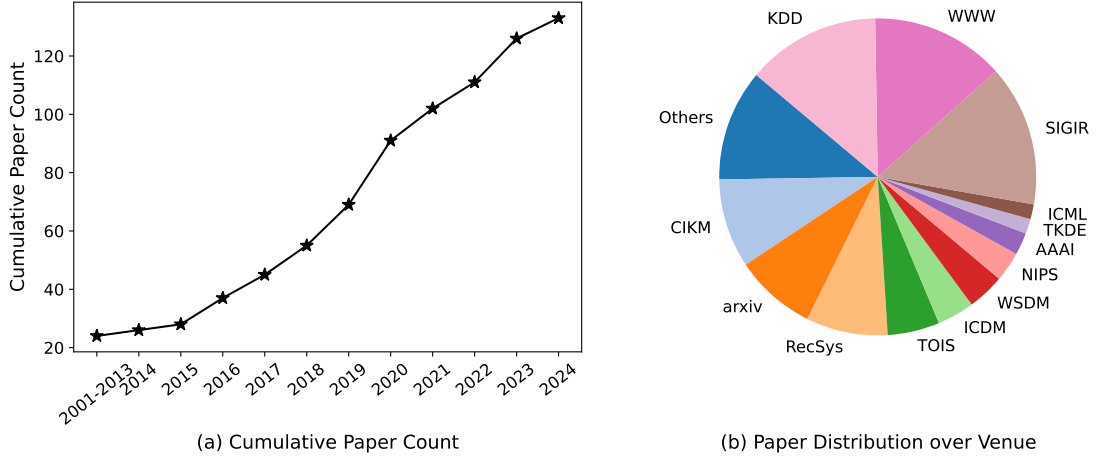


Fig. 2. Distributions of the reviewed papers of retrieval methods in recommender systems over years (a) and over venues (b).

is not a specific algorithm but a broad family of algorithms, including Item2Vec [14], YouTube DNN [43], EGES [159] of Alibaba, and Pinsage [192] of Pinterest, e.t.c. These models, which vary from two-tower to graph-embedding structures, aim to learn comprehensive representations for users and items. Given the extensive research conducted, it is timely to assess the current landscape, learn from existing methods, and identify directions for future research. While there are comprehensive surveys on the ranking stage [66, 173, 203], the retrieval stage lacks systematic reviews. This oversight may stem from the greater emphasis traditionally placed on the ranking stage, which, as the final stage in recommender systems, interacts directly with users and has a more immediate impact on business objectives. However, the retrieval stage is also crucial as it sets the stage for all subsequent processes. Without effective retrieval, even the most advanced follow-up stages cannot perform optimally.

1.1 A Taxonomy and Major Content

To our knowledge, this is the first survey that comprehensively summarizes retrieval-related efforts in recommender systems. While there have been several surveys on recommender systems, none have specifically focused on the retrieval stage. Batmaz et al. [15] provide a thorough review of deep learning based recommendation methods, primarily focusing on autoencoders, recurrent neural networks (RNNs), and convolutional neural networks (CNNs) for recommendations. Similarly, Zhang et al. [200] offer a detailed overview of current efforts in deep learning based recommender systems and discuss some unresolved issues. Xu et al. [183] propose a unified framework for search and recommendation and mention some retrieval models, but their discussion is scattered and lacks a focus on the retrieval stage. In contrast, our work provides a comprehensive overview of retrieval methods in recommender systems under a unified framework.

We collect over 100 research papers on retrieval methods in recommender systems. In conducting our literature search, we utilize DBLP and Google Scholar as the primary search engines, using keywords such as *candidate matching*, *candidate retrieval*, *large-scale recommendation*, and *collaborative filtering* in the titles and abstracts to filter relevant papers from top-tier conferences and journals in data mining, machine learning, recommender systems, and artificial intelligence. These include, but are not limited to, KDD, ICDM, WWW, SIGIR, ICML, NIPS, AAAI, WSDM, CIKM, RecSys, TKDE, and TOIS, covering the period from 2001 to 2024, as shown in Figure 2. We traverse the citation graph of the identified papers and incorporate pertinent studies. In addition to reviewing published papers, we also screen

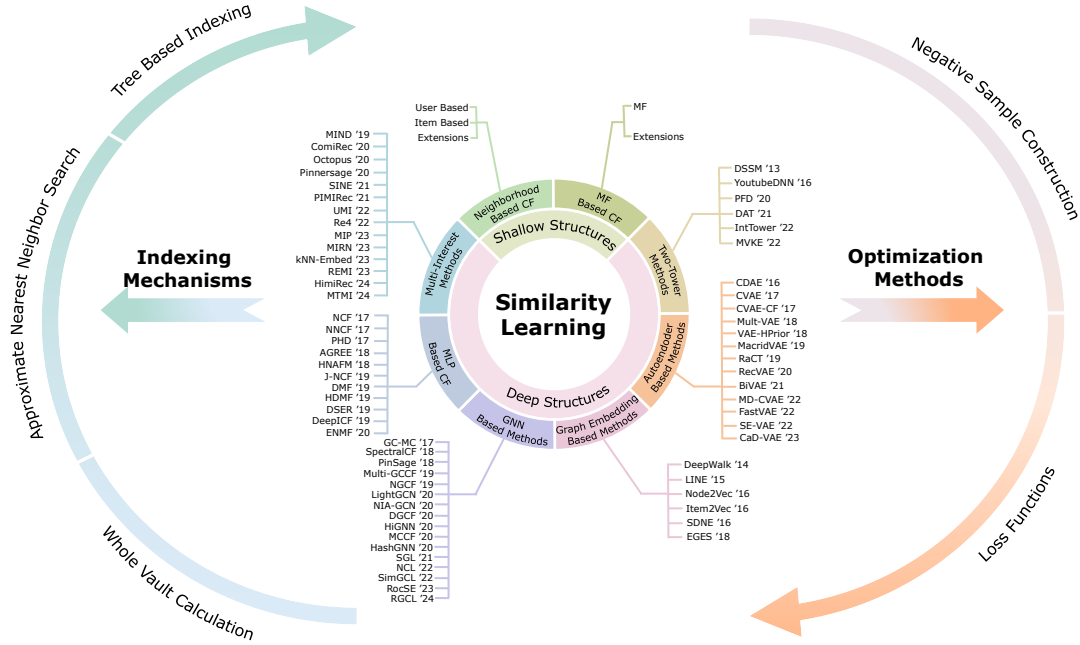


Fig. 3. Taxonomy for retrieval methods in recommender systems.

preprints on arXiv, identifying those with novel and intriguing ideas to provide a more comprehensive perspective. Based on this corpus, we summarize the efforts made to achieve precise and efficient personalized retrieval into three main areas, as illustrated in Figure 3:

- **Similarity Learning.** Many studies focus on improving the representation of users and items to enhance similarity learning. Traditional methods, based on CF and MF [84, 115, 137], use shallow structures for this purpose. Recently, there has been a growing trend towards using deep structures [43, 159, 192] to better learn the representations and similarity relationships of users and items.
- **Indexing Mechanisms.** In the retrieval stage, indexing schemes are essential for effectively organizing and retrieving large-scale items. Some efforts have been directed towards improving indexing mechanisms, such as tree based methods that use a tree structure as an index [212, 213, 217]. Deep Retrieval [55] uses a matrix for indexing, enabling efficient item clustering and retrieval.
- **Optimization Methods.** Various learning strategies have been developed to optimize the training of retrieval methods, particularly in negative sample construction. Users typically interact with a small subset of items compared to the vast number available in real systems, making the lack of reliable negative data a challenge for learning from implicit feedback. This includes various negative sampling strategies [64, 133], which select a subset of missing data as negative instances, and whole-data based strategies [45, 69, 98] that treat all missing data as negative instances.

The aim of this survey is to thoroughly review the literature on retrieval methods in recommender systems and discuss the current state of retrieval in the industry, as well as future directions. This survey provides researchers and practitioners interested in recommender systems with a general understanding of the latest developments in the field of retrieval for recommendation. The key contributions of this survey are summarized as follows:

- **First Comprehensive Review.** This review addresses the deficiency in summarizing retrieval-related efforts in recommender systems. We categorize these efforts into three main areas: improving similarity learning between user and item through both shallow and deep structures, enhancing indexing mechanisms for efficient large-scale retrieval, and optimizing training methods of retrieval. To the best of our knowledge, this is the first comprehensive review on the retrieval methods for recommendation, which not only highlights the progress achieved but also identifies areas for future research and development.
- **Extensive Benchmarking Experiments.** We provide a comprehensive set of benchmarking experiments, covering various retrieval methods on three public datasets. As the study of retrieval methods is deeply rooted in practical applications and faces numerous real-world challenges, we summarize online performance statistics from published papers, providing valuable insights.
- **Case Study on Industrial Practices of Retrieval.** Additionally, we provide a case study on the current industrial practices of the retrieval stage at a specific company, covering the entire retrieval process and online serving, along with key insights and challenges encountered.

The remaining of this article is structured as follows: Section 2 introduces the background of retrieval in recommender systems, including problem formulation, basic retrieval strategies and input data for retrieval. Sections 3 and 4 detail the efforts that focus on similarity learning using shallow and deep structures, respectively. Section 5 discusses improvements in indexing mechanisms for efficient retrieval. Section 6 covers optimization methods of retrieval, including key aspects of model learning, such as widely-used loss functions and different negative sampling strategies. Section 7 presents commonly used evaluation metrics for both offline and online settings. Section 8 summarizes commonly used datasets and conducts detailed experiments on three public datasets to assess the effectiveness of various methods. Section 9 presents a specific case study of current industrial practices in the retrieval stage at a specific company. We discuss open challenges and suggest promising future research directions in Section 10, and conclude the survey in Section 11.

2 BACKGROUND

In this section, we first formulate the task of retrieval methods. Then we discuss the existing retrieval strategies from a broad perspective, including non-personalized and personalized retrieval. Next, we cover the common input data or information used in retrieval methods, including the user-item rating matrix and rich side information. Finally, we introduce the multi-channel retrieval, which is widely used in real-world recommender systems.

2.1 Problem Formulation

We denote the universal user and item set as $\mathcal{U}$ and $\mathcal{I}$, respectively. For a given user u , the objective of the retrieval method is to identify all potentially relevant items from the vast candidate pool $\mathcal{I}$. Generally, the retrieval method can be formulated as:

$$C(u, \mathcal{I}) = \{i \in \mathcal{I} \mid s(u, i) > \theta\}, \quad (1)$$

where $C(u, \mathcal{I})$ is the candidate set for user u , $s(u, i)$ is a scoring function that measures the relevance of item i to user u , and θ is a threshold for relevance. Unlike the ranking phase, which deals with a smaller candidate set, the retrieval stage

may involve filtering through millions of items. Consequently, efficiency becomes a critical concern for models used in this stage. As a result, most retrieval methods employ a dual-tower architecture where user and item representations are learned separately. Formally, given user $u \in \mathcal{U}$ and item $i \in \mathcal{I}$, the process can be abstracted as:

$$s(u, i) = f(\phi_u(u), \phi_i(i)), \quad (2)$$

where $\phi_u : \mathcal{U} \rightarrow \mathcal{H}$ and $\phi_i : \mathcal{I} \rightarrow \mathcal{H}$ represent the mappings of user space $\mathcal{U}$ and item space $\mathcal{I}$ to a new space $\mathcal{H}$, respectively. f denotes the similarity measure between user and item, utilizing metrics like inner product or cosine similarity. For any user-item pair (u, i) , $s(u, i)$ provides a score reflecting the similarity between u and i . This score enables the ranking of all items in the corpus $\mathcal{I}$ according to their predicted similarity scores. By ranking the items based on these scores, the retrieval method can efficiently generate a set of items, forming the retrieved set for user u .

2.2 Retrieval Strategy

Before exploring specific retrieval methods, it is important to distinguish between retrieval strategies and methods, and clarify the relationship between two of them, as they operate on different levels. The retrieval strategy serves as the high-level concept that guides the design of various retrieval methods. Generally, as illustrated in Figure 4 (a) and Figure 4 (b), retrieval strategies can be broadly categorized into (1) non-personalized retrieval and (2) personalized retrieval. Non-personalized retrieval primarily aims at identifying and presenting trending topics. Though such an approach might not directly align with a user's specific interests, there is a high likelihood of user clicks due to the popularity of the content. In contrast, personalized retrieval tailors its recommendations to match user preference, significantly enhancing user engagement and retention. This category includes varied strategies like item-to-item (I2I) and user-to-item (U2I), which will be detailed further. These strategies provide general guidance for the design of different retrieval methods. Understanding this distinction helps clarify how different approaches are applied in practice.

2.2.1 Non-Personalized Retrieval. Non-personalized retrieval plays a pivotal role in the retrieval stage without tailoring to individual user preference, as shown in Figure 4 (a). One common strategy is promoting popular items, which is inspired by the 'wisdom of the crowd' [145]. It is particularly effective for new users or in the absence of sufficient user interaction data [10]. Another strategy focuses on less mainstream items, aiming to diversify the user's preference and unearth user potential interest. Newly released item promotion [26] is also a widely used strategy, which pushes the latest item to users, fueling exploration and maintaining a fresh and dynamic environment [52]. Besides, the platform would design various operational algorithms to serve specific business goals or events, guiding users towards particular topics or items. Each of these strategies serves to enhance user engagement without personalization, necessitating the use of more advanced strategies and techniques.

2.2.2 Personalized Retrieval. Personalized retrieval recommends items that align with user interests. While popular items may draw broad attention, they do not always match individual user preferences. Hence, personalized retrieval aims to cater to users' dynamic information needs and maintain their stickiness. As is shown in Figure 4 (b), U2I, U2I2I (often shortened as I2I for simplicity), and U2U2I are three popular personalized retrieval strategies, where 'U' stands for the user, 'I' stands for the item, and '2' is short for 'to' that denotes the process of seeking similarity or connection. For instance, U2I directly correlates the target user with items he/she might like; U2I2I identifies items similar to those the target user has previously interacted with; and U2U2I finds users with similar tastes to suggest items for the target user. Each path introduced above offers a unique strategy to discover and recommend items that user might be interested in.

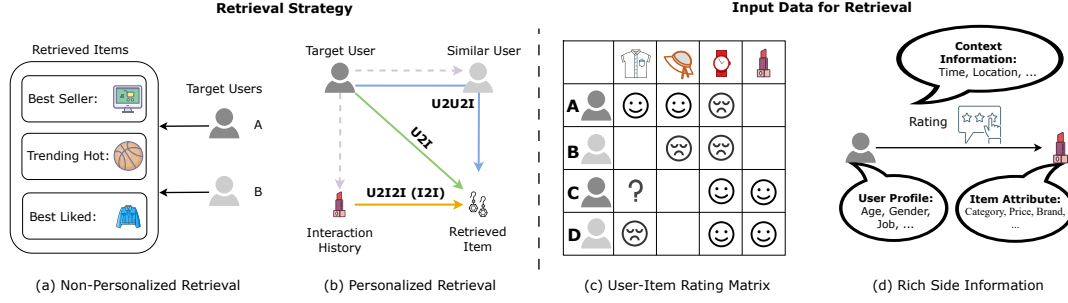


Fig. 4. An illustration of different retrieval strategies and input data for retrieval. Figure (a) illustrates non-personalized retrieval, a strategy that offers the same recommendation list to different users, such as ‘trending hot’, without tailoring to individual user interests. Figure (b) depicts personalized retrieval, which includes three retrieval strategies: U2I, U2U2I, and U2I2I (I2I). Figure (c) shows the user-item rating matrix, which is the core data or information used in retrieval methods. Figure (d) presents information beyond the user-item rating matrix, commonly involving side information such as user profiles, item attributes, and context information.

Additionally, recommender systems often use collaborative filtering (CF) and content based (CB) classifications. CF operates on the principle of gauging similarity based on user interaction data, typically through a rating matrix reflecting user interactions with items. This approach assumes that users who have agreed in the past are likely to agree in the future. On the other hand, CB methods focus on item attributes or user profiles to determine similarity. It analyzes item features and recommend items similar to those a user has shown interest in before, based on the content of the items themselves. Both CF and CB offer distinct ways to measure similarity, with CF leveraging user behavior patterns across the community, and CB concentrating on the item attributes and user profiles. The combination of these strategies allows for a more nuanced approach to personalized retrieval, enhancing user experience by connecting individuals with content that resonates with their interests and past interactions.

2.3 Input Data for Retrieval

In this section, we will introduce the common input data or information used in retrieval methods, which includes the user-item rating matrix and rich side information. We categorize the side information into three aspects: user profile, item attributes, and contextual information.

2.3.1 User-Item Rating Matrix. The user-item rating matrix, denoted as R , is the fundamental input data for retrieval, capturing user preferences towards various items. Each entry R_{ij} within the matrix indicates the preference of user i for item j ; if $R_{ij} > 0$, it suggests a positive preference. Preferences can be recorded directly through explicit ratings or indirectly through binary indicators that reflect actions such as clicks, views, or purchases. In Figure 4 (c), we show the user-item rating matrix, where positive feedback is indicated by smiley faces and negative feedback by sad faces. It is important to note that the matrix is typically sparse, as the known preferences are usually limited, with many entries unspecified. We usually regard these unspecified (i.e., not interacted) entries as negative samples.

2.3.2 Rich Side Information. In addition to the user-item rating matrix, three types of side information play crucial roles in enhancing retrieval methods in recommender systems [141], as illustrated in Figure 4 (d). One vital source is the user profile, which might include demographics information, such as a user’s gender, age, and hobbies. Item attributes also provide rich information, detailing properties of the items like their category or content. Contextual

information provides insights into the circumstances under which user interactions with items occur [11]. Commonly included in this category are timestamps, which track when interactions like ratings or purchases take place [83, 180]. Additionally, other relevant details like a user’s location during an app download [21] or their hunger status when rating a food menu [121] can also enrich the understanding of user-item interactions, further tailoring the retrieval process to specific user conditions or preferences.

Additionally, in recent years, user-generated content (UGC) has become increasingly prevalent on UGC platforms, enriching item attributes. Tags, for instance, are short textual labels that users freely assign to items without being limited to a predefined list of categories [135]. These tags can describe the characteristics of an item or express users’ feelings, serving as a valuable source of data. Reviews and comments published by users constitute another crucial source of side information that has been successfully leveraged to improve the performance [87].

Empirical evidence indicates that incorporating rich side information yields substantial improvements over using only the user–item rating matrix. For instance, Tso-Sutter et al. [154] report that fusing tag information with ratings increases Recall@10 from 0.32 to 0.34 compared to pure user and item based collaborative filtering on the Last.fm [9] dataset. SocialMF [75] incorporates social trust propagation into matrix factorization, reducing RMSE from 1.175 to 1.075 on the Epinions dataset [1] and from 0.878 to 0.821 on Flixster [2] relative to basic matrix factorization. With respect to contextual data, Koren [83] demonstrate that adding temporal dynamics reduces RMSE from 0.913 to 0.897 on the Netflix movie-rating dataset [18] compared to a static SVD++ baseline. Details on datasets appear in Section 8, and metric definitions appear in Section 7.

2.4 Multi-Channel Retrieval

Multi-channel retrieval is widely used in recommender systems, which employs various independent retrieval methods to individually retrieve different subsets of items [43, 57]. As is shown in Figure 1 (b), these retrieved item subsets are then aggregated to form a comprehensive candidate pool for downstream ranking stages. The primary motivation is to maximize the coverage of users’ diverse interests and improve the recall rate [200]. By using different retrieval methods, multi-channel retrieval captures a wide range of potential user preferences and enhances the overall performance of recommender systems.

The choice of retrieval channel is highly dependent on the specific applications. For example, the ‘trending list’ and ‘interest tags’ are essential for timely platforms like news streaming, while ‘friend favorites’ is more important for social media platforms. The number of items to be retrieved (i.e., top- K) can vary for different channels, and would be typically optimized through offline evaluation and online A/B testing [71].

2.5 Comparison with Multi-Stage Retrieval in Ad Hoc Search

Multi-stage cascaded pipelines are also well established in ad hoc search [38, 42]. Clarke et al. [42] introduce a framework that quantifies efficiency–effectiveness trade-offs in multi-stage query systems without relying on relevance judgments. Chen et al. [38] propose a cost-aware cascade ranking model that jointly optimizes per-stage cutoffs to balance latency against retrieval quality in a static document index.

Although both ad hoc search and recommender system pipelines employ the same core principle, i.e., progressively narrowing a large corpus through increasingly expensive stages, they differ along several key dimensions:

- **Personalization and dynamics.** Ad hoc search typically serves identical results to all users over a fixed index, whereas recommender systems have to tailor candidate sets to individual user profiles and adapt continuously as new items and interactions arrive.

- **Data freshness and indexing.** Ad hoc indices are updated at regular intervals, making feature extraction relatively stable; recommender pipelines often require real-time or near-real-time indexing to incorporate the latest user behavior and item metadata.
- **Feature representations.** Early ad hoc stages use term-based statistics (e.g., BM25 [134]), while the retrieval stage in recommendation leverages learned embeddings that combine user history, content attributes, and contextual signals.

3 SIMILARITY LEARNING USING SHALLOW STRUCTURES

In Sections 3 and 4, we discuss efforts focused on accurate user-item similarity learning, thereby enhancing effectiveness. This section centers on similarity learning using shallow structures, which primarily revolves around the concept of collaborative filtering (CF). CF, as the name suggests, leverages collective feedback, evaluations, and opinions to help users filter through vast amounts of information, which is a cornerstone technique in the retrieval stage of recommender systems. These approaches can be divided into two categories: (1) neighborhood based [137] and (2) matrix factorization (MF) based [84, 115] methods. In this section, we will explore these two types of traditional retrieval methods in details.

3.1 Neighborhood Based Collaborative Filtering Methods

The neighborhood based CF method (also called memory based CF methods in some literature [37, 142]) is a fundamental subset of collaborative filtering techniques used in the retrieval stage. These methods operate by clustering users or items based on similarities in their interactions and preferences, thereby forming ‘neighborhoods’. There are primarily two types of neighborhood based CF methods: user based and item based, each with its distinct processes, advantages, and application contexts.

3.1.1 User Based CF. This approach focuses on finding similar users to the target user. Similarity between users is typically calculated using measures such as cosine similarity or pearson correlation coefficient based on their ratings in a user-item rating matrix [22, 136]. The similarity between user u and user v can be represented as:

$$s(u, v) = \frac{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{uv}} (r_{ui} - \bar{r}_u)^2} \sqrt{\sum_{i \in I_{uv}} (r_{vi} - \bar{r}_v)^2}}, \quad (3)$$

where I_{uv} is the set of items rated by both users u and v , r_{ui} is the rating of user u for item i , and $\bar{r}_u$ is the average rating of user u . Once similar users are identified, the system recommends items that these similar users have liked but the target user has not yet interacted with. User based CF can provide highly personalized recommendations since it directly leverages user behaviors and preferences. However, its scalability [79] is a significant issue due to the computational complexity of comparing each user with every other user, especially in systems with a large number of users. It also suffers from the cold-start problem for new users with insufficient interaction data.

3.1.2 Item Based CF. Item based CF focuses on calculating item similarity based on the patterns of users who have interacted with them [104, 137]. The similarity between item i and item j can be calculated as:

$$s(i, j) = \frac{\sum_{u \in U_{ij}} (r_{ui} - \bar{r}_i)(r_{uj} - \bar{r}_j)}{\sqrt{\sum_{u \in U_{ij}} (r_{ui} - \bar{r}_i)^2} \sqrt{\sum_{u \in U_{ij}} (r_{uj} - \bar{r}_j)^2}}, \quad (4)$$

where U_{ij} is the set of users who have rated both items i and j , r_{ui} is the rating of user u for item i , and $\bar{r}_i$ is the average rating of item i . Recommendations to a user are made by identifying items similar to those the user has already liked or interacted with, assuming that users will prefer items similar to their previous interests. Item based CF tends to be more

scalable than user based CF, as the item-item similarity matrix is typically more stable and changes less frequently than user-user similarities [17, 137, 146]. It also mitigates the cold-start problem for long-tail users with limited interactions to some extent, but still struggles with completely cold-start users. Item based CF is particularly effective in systems where item relationships are more pronounced and stable, such as movie or book recommendation systems where items have longer lifespans and well-defined features, but it is not suitable in cases where user tastes are highly diverse or when new items frequently enter the system.

Both user based CF and item based CF enhance the retrieval process by incorporating the vast amount of user-item interactions. However, the choice between them, or the decision to integrate both approaches, depends largely on the specific requirements, scalability considerations, and the nature of user interactions within the recommendation platform. Modern recommender systems often employ hybrid models that combine the strengths of user based CF and item based CF to overcome their respective limitations.

3.1.3 Extensions to Neighborhood Based CF. Traditional neighborhood based CF methods, like those described above, primarily utilize direct pairwise interactions between users and items to make recommendations. These are either based on user-user or item-item similarities derived directly from the user-item rating matrix. The Swing algorithm [188] introduces a quasi-local structure that goes beyond these direct interactions by examining the relationship among users and items in a more comprehensive manner. The Swing algorithm is detailed using the concept of a swing score, which measures the relationship strength between two items based on their shared interactions with users. Specifically, the swing score between item i and item j is defined in equation 5, where U_i is the set of users who have clicked item i , and I_u is the set of items that user u has clicked. The parameter α serves as a smoothing coefficient to manage the influence of highly interactive user-item pairs. By utilizing such quasi-local structures, Swing can uncover more complex and subtle patterns in user-item interactions, leading to a richer and more accurate collaborative filtering result.

$$s(i, j) = \sum_{u \in U_i \cap U_j} \sum_{v \in U_i \cap U_j} \frac{1}{\alpha + |I_u \cap I_v|} \quad (5)$$

Additionally, some studies have built upon the user-item rating matrix by using side information to refine similarity calculations [141]. Melville et al. [118] first propose filling missing values in the user-item rating matrix using item side information such as titles and genres. Another category of approaches focuses on using tags as side information [51, 139]. Tso-Sutter et al. [154] develop a method to blend user-user or item-item similarities based on both tags and ratings within a neighborhood based CF framework. To combat the inherent noisiness of tags, schemes to enhance tag reliability have been introduced, such as assigning different weights to different tags and integrating these with conventional neighborhood based CF approaches [100].

3.2 Matrix Factorization Based Collaborative Filtering Methods

3.2.1 MF Based CF. Matrix factorization (MF) based Collaborative Filtering (CF) methods [84], also referred to as latent factor models [82, 142], play a pivotal role in the retrieval stage. Originating from the need to effectively map both users and items into a latent factor space, these methods estimate user-item interactions, typically ratings, by calculating the dot product of their latent factors. The essence of MF involves decomposing the user-item rating matrix R (of shape $M \times N$, where M is the number of users and N is the number of items) into two lower-dimensional matrices: U (of shape $K \times M$, representing latent user preferences) and V (of shape $K \times N$, representing latent item attributes), aiming to approximate the original rating matrix R with the product $U^T \times V$ as shown in equation 6. Optimization of these methods, given the sparsity of the rating matrix and computational constraints, is typically achieved through

techniques such as stochastic gradient descent, complemented by regularization terms to mitigate overfitting. We use the same notation in previous works [141] and as is shown in equation 7, U and V represent matrices of latent factors, where U^* and V^* denote their optimal values determined through the optimization process. Each column vector U_i in U corresponds to the latent factors associated with user i , while V_j in V corresponds to the latent factors for item j . The indicator function I_{ij} equals 1 if $R_{ij} > 0$ and 0 otherwise. The Frobenius norm of a matrix is represented as $\|U\|_F$, and λ_U and λ_V are regularization parameters used to prevent overfitting in the model.

$$R \approx U^T \times V. \quad (6)$$

$$U^*, V^* = \arg \min_{U, V} \left\{ \frac{1}{2} \sum_{i=1}^M \sum_{j=1}^N I_{ij} (R_{ij} - U_i^T V_j)^2 + \frac{\lambda_U}{2} \|U\|_F^2 + \frac{\lambda_V}{2} \|V\|_F^2 \right\} \quad (7)$$

3.2.2 Extentions to MF Based CF. MF based CF gained prominence during the Netflix Challenge [147], highlighting their capability to handle large and sparse datasets by uncovering complex, latent relationships within the data. To expand upon the foundational MF approach, advanced variations like SVD++ [82] were developed. SVD++ not only considers the explicit ratings that users provide but also integrates implicit feedback—such as browsing or purchase history—to enhance the model’s predictive accuracy and relevance. This integration helps capture a more complete picture of user behavior and preferences, greatly enriching the algorithm’s applicability.

Further extending the capabilities of traditional MF, Factorization Machine (FM) [131] incorporates additional side information, which MF does not typically include. Unlike traditional MF, which focuses on learning latent representations for user and item IDs, FM is able to model interactions among more available features, providing a powerful framework capable of operating within highly sparse data environments and across different domains, where data interconnectivity plays a critical role. Time based dynamics are considered in methods like TimeSVD++ [83], which specifically incorporates temporal information to improve predictions over time. Additionally, SLIM [120] is a simple linear model that merges the benefits of neighborhood CF and latent factor models. These enhancements and innovations in MF based methods underscore their vital roles in the evolution of retrieval methods, which offer a scalable solution that delivers quick and precise predictions even for large databases [20]. The methods discussed so far primarily use shallow structures for user-item similarity learning. In Section 4, we will introduce more recent works that leverage deep structures, exploring how deep learning techniques are leveraged to further improve the effectiveness of retrieval methods.

4 SIMILARITY LEARNING USING DEEP STRUCTURES

Recently, with the advent of deep learning, there has been a significant shift towards more sophisticated and deep structures for user-item similarity learning. Modern recommender systems are increasingly adopting embedding based retrieval (EBR). EBR transforms the retrieval problem into a nearest neighbor search within a vector space, where both users and items are represented as points. This structure allows EBR to incorporate rich user and item features into the representation learning process, considering a broader range of features beyond mere user and item IDs. We classify these methods into six categories based on their structure as is shown in Figure 5 (i.e., tow-tower methods, autoencoder based methods, graph embedding based methods, graph neural network based methods, MLP based collaborative filtering methods, and multi-interest methods), and we will now discuss each category in details.

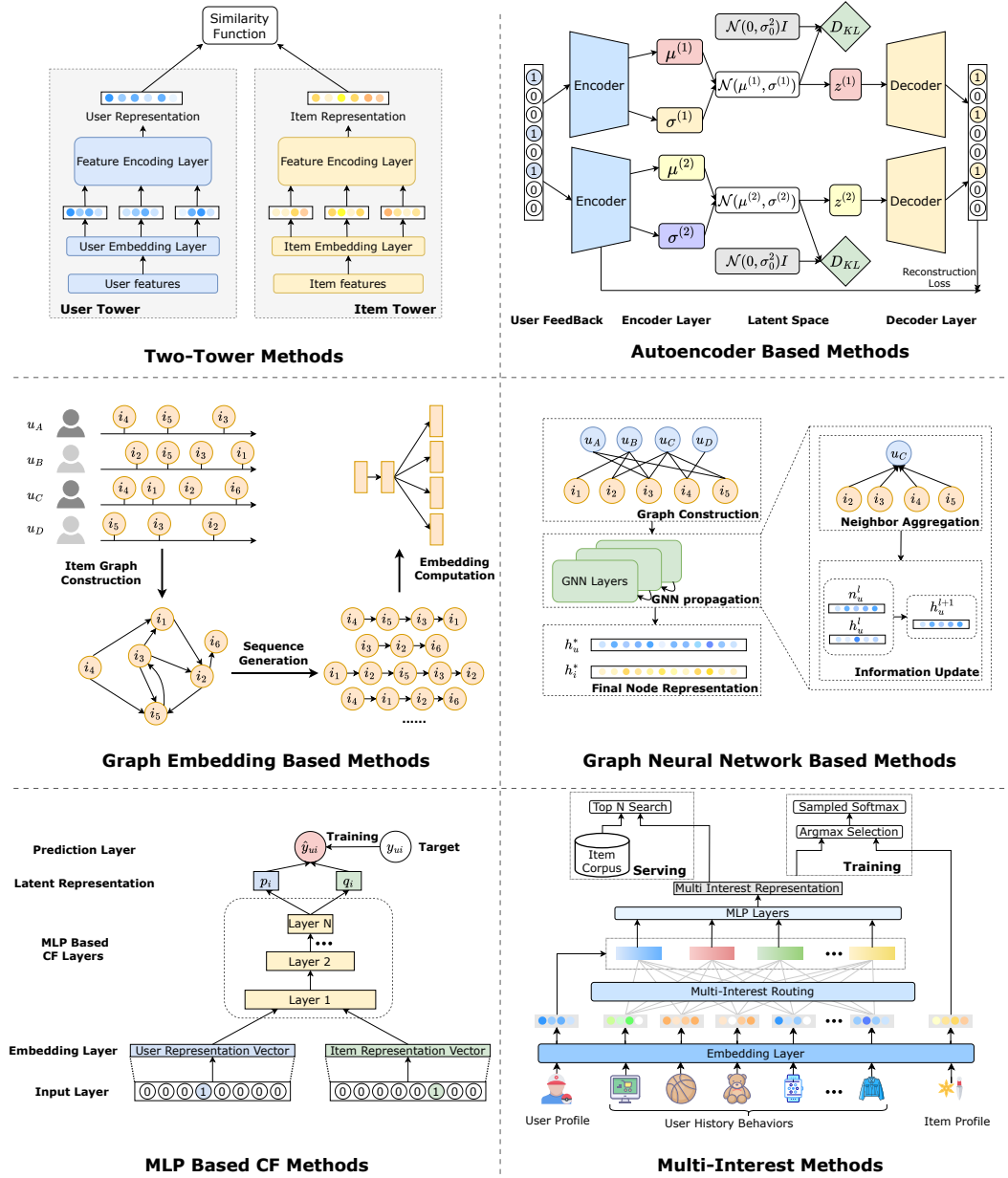


Fig. 5. The architecture of deep retrieval methods focused on similarity learning.

4.1 Two-Tower Methods

The two-tower methods represent a state-of-the-art approach in the retrieval stage, offering a balance between prediction accuracy and inference efficiency [92]. In this architecture, each user u and item i is independently mapped to a

representation via two parallel sub-networks: $\phi_u(u; \theta_u)$ and $\phi_i(i; \theta_i)$, where ϕ_u and ϕ_i denote the user tower and item tower, respectively, and θ_u, θ_i are their parameters. Taking the user tower as an example, user features are first passed through an embedding layer, where user feature embeddings are obtained via an embedding look-up operation [202]. These embeddings are then processed through multiple fully connected layers to refine the representation. The final output of the two-tower model is determined by the inner product of the user and item representations.

$$s(u, i) = \langle \phi_u(u; \theta_u), \phi_i(i; \theta_i) \rangle. \quad (8)$$

This architecture decouples the user and item modeling, which is highly efficient and significantly reduces inference latency and computational resources. Classic implementations of two-tower methods in industry include Microsoft’s DSSM [72], Google’s YouTubeDNN [43], and Airbnb’s personalized embeddings [57], leading to significant performance improvements. The two-tower methods mentioned above, while efficient, often face limitations in adequately modeling user-item interactions, which can impede model performance. To overcome such challenges, advanced techniques have been developed to enhance the depth of interaction between the user and item side, which we will further detail in future directions in Section 10.

4.2 Autoencoder Based Methods

Autoencoder based methods, particularly Variational Autoencoders (VAEs) [81], are increasingly utilized in the retrieval stage of recommender systems to address challenges like data sparsity and the complexity of user-item interactions. These methods excel in generating and representing complex data structures. VAEs operate by assuming that each observed interaction vector $\mathbf{r}_u \in \{0, 1\}^N$ is generated from a latent variable $\mathbf{z}$ following some probability distribution, such as standard Gaussian distribution. They consist of two main components: an encoder and a decoder. The encoder network $q_\phi(\mathbf{z} | \mathbf{r}_u)$ approximates the true posterior distribution of the latent variables given the input data, effectively capturing the underlying data structure:

$$\boldsymbol{\mu}_u = \mu(\mathbf{r}_u), \quad \log \sigma_u^2 = \log \sigma^2(\mathbf{r}_u), \quad (9)$$

$$\mathbf{z}_u = \boldsymbol{\mu}_u + \sigma_u \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}). \quad (10)$$

The decoder then reconstructs $\mathbf{r}_u$ from the latent representation, as shown in equation 11, where $\sigma(\cdot)$ denotes the element-wise sigmoid function. It aims to match the original input as closely as possible.

$$\hat{\mathbf{r}}_u = \sigma(\text{Decoder}(\mathbf{z}_u)), \quad (11)$$

The effectiveness of VAEs in the retrieval stage of recommender systems is showcased by their ability to learn comprehensive user preferences and item characteristics through deep neural networks. Wu et al. [174] introduce CDAE, which incorporates a user-specific bias into an autoencoder framework. CDAE represents a generalization of various existing CF techniques. Mult-VAE [99] sparks interest in VAE for CF by showcasing its superiority over other benchmarks. Simultaneously, Lee et al. [86] explore the VAE framework for CF with an emphasis on conditional and joint VAE formulations to integrate auxiliary user data. CVAE [93] and MD-CVAE [216] enhance collaborative latent variable modeling by incorporating item content information. Karamanolakis et al. [78] apply VAE to personalized recommendations, delving into user-dependent priors. More recent efforts by Kim and Suh [80] evaluate VAEs enhanced by VampPrior [152]. MacridVAE [112] and CaD-VAE [163] focus on modeling the latent factors behind interactions to accurately capture the distribution of latent variables. RaCT [110] introduces an actor-critic reinforcement learning technique to optimize VAEs, while RecVAE [138] examines various regularization methods to refine VAE’s performance

in CF. BiVAE [153] treats users and items symmetrically and proposed bilateral user and item based inference models to autoencode users and items under a unified framework. FastVAE [34] improves the efficiency of the model using an inverted multi-index technique. SE-VAE [40] advances latent variable modeling by integrating a multi-expert system with stochastic expert selection, enhancing the model's adaptability and accuracy.

4.3 Graph Embedding Based Methods

Graph embedding based methods have been pivotal in enhancing the retrieval process in recommender systems. These methods extend the principles of word embeddings, such as Word2Vec [119], to user-item interaction graphs. These methods learn a mapping that assigns each node v in the graph $G = (\mathcal{V}, \mathcal{E})$ to a vector $\Phi(v)$. They optimize a neighborhood-preserving objective of the form:

$$\min_{\Phi} \sum_{v \in \mathcal{V}} \sum_{c \in \mathcal{N}(v)} -\log \Pr(c | \Phi(v)), \quad \Pr(c | \Phi(v)) = \frac{\exp(\Phi(c)^\top \Phi(v))}{\sum_{u \in \mathcal{V}} \exp(\Phi(u)^\top \Phi(v))}, \quad (12)$$

where $\mathcal{N}(v)$ denotes a chosen context set for node v (e.g., adjacent nodes, nodes co-occurring in random walks, or nodes sharing side-information). Minimizing this loss ensures that frequently co-occurring or closely related nodes receive similar embeddings, while unrelated nodes are pushed apart. Item2Vec [14] is a direct adaptation of Word2Vec to items, treating the user behavior sequences of items (like products or songs) as sentences to uncover similarities based on co-occurrence. This enables better recommendations by capturing contextual similarities among items. For example, Airbnb successfully applied Item2Vec [57] to personalize user experiences in online accommodations. DeepWalk [125] uses random walks on graphs to generate node sequences, learning representations that reflect the social structure of interactions. Node2Vec [58] enhances this approach by balancing breadth-first and depth-first sampling, capturing diverse connectivity patterns in the graph. Further refining this, EGES [159] integrates side information (like user demographics or item attributes) into the embeddings, enriching the model's contextual understanding. LINE [151] preserves both direct connections and neighbor similarities, ensuring embeddings reflect both explicit and implicit relationships. SDNE [155] uses autoencoders to learn node representations, adeptly capturing nonlinear relationships within complex, hierarchical structures. All these models underscore the importance of graph embeddings in understanding and leveraging complex user-item interaction patterns.

4.4 Graph Neural Network Based Methods

Recent advancements have incorporated Graph Neural Network (GNN) techniques into the retrieval stage of recommender systems [19, 62, 109, 144], known for their efficacy in handling complex relational data. To understand the distinction between graph embedding based methods and GNN based methods, it is essential to first explain their foundational differences. Graph embedding based methods typically involve deriving item sequences from the graph and then generating embeddings from these sequences. These methods focus on capturing the structural properties and proximities within the graph to represent items in a continuous vector space. In contrast, GNN based methods utilize graph convolution operations, which involve iterative message passing and aggregation across the graph's nodes and edges. In a typical message-passing paradigm, for each node v at layer k , one computes

$$\begin{aligned} m_v^{(k)} &= \text{AGG}(\{h_u^{(k-1)} : u \in \mathcal{N}(v)\}), \\ h_v^{(k)} &= \text{UPDATE}(h_v^{(k-1)}, m_v^{(k)}), \end{aligned} \quad (13)$$

where $\mathcal{N}(v)$ is the set of neighbors of v , $h_v^{(k)}$ is the representation of node v at layer k , $\text{AGG}(\cdot)$ defines a pooling or aggregation function over neighbors (e.g., mean, sum, or attention), and $\text{UPDATE}(\cdot)$ is a function—often a linear transformation followed by a nonlinearity—that integrates the previous representation with the aggregated message. GNNs excel at capturing intricate user-item interactions, optimizing the retrieval process by exploiting high-order relationships. Figure 5 illustrates the pipeline for applying GNNs in retrieval.

- **Graph Construction.** Applying GNNs directly to user-item graphs [19, 62, 143, 207] can face challenges due to sparse connections or large scale. Enhancements such as adding connections [109, 144] or virtual nodes [97, 165] improve graph connectivity and retrieval quality. Techniques like Multi-GCCF [144] and DGCF [109] enhance graphs by linking two-hop neighbors, while HiGNN [97] clusters similar entities to enhance user-item graphs. For better efficiency, sampling methods like those used in Multi-GCCF [144] and PinSage [192] help process large-scale graphs by balancing the retention of meaningful information and computational demands.
- **Neighbor Aggregation.** This step propagates information through the graph. Methods vary from mean-pooling [19, 148] to more sophisticated methods like attention mechanisms [113, 166] that weigh neighbors based on relevance. PinSage [192] uses normalized visit counts to determine neighbor importance.
- **Information Update.** Updating node representations is crucial. Some methods fully replace node representations with aggregated information [19, 62], while others combine original attributes with neighbor information through pooling or transformations [88, 192]. LightGCN [62] simplifies this process by omitting non-linearities, enhancing performance and efficiency.
- **Final Node Representation.** After multiple layers of aggregation and updating, the final node representation is used for predictions. Methods like weighted pooling and concatenation integrate information from different layers, leveraging the unique characteristics captured at each depth.

Recent studies [150, 189, 194] have demonstrated that contrastive learning in GNN based retrieval significantly improves robustness. Yu et al. [194] argue that costly graph perturbations are unnecessary; instead, generating contrastive views by injecting uniform noise into the embedding space achieves superior performance and efficiency. Subsequent research [150, 189] has further enhanced model resilience by applying structure denoising and adversarial embedding perturbations. Although GNN based retrieval methods show remarkable results in understanding and leveraging complex user-item interaction patterns, enhancing their efficiency for large-scale retrieval remains a critical problem.

4.5 MLP Based Collaborative Filtering Methods

Unlike conventional collaborative filtering techniques that often rely on linear models and explicit factorization methods, neural collaborative filtering (NCF) [64] uses an MLP to model non-linear user-item interactions. Concretely, let $\mathbf{e}_u, \mathbf{e}_i$ be the embedding vectors for user u and item i . NCF concatenates these embeddings as

$$\mathbf{h}_{u,i}^{(0)} = [\mathbf{e}_u \parallel \mathbf{e}_i], \quad (14)$$

and then applies L hidden layers:

$$\mathbf{h}_{u,i}^{(\ell)} = \text{Activation}(\mathbf{W}^{(\ell)} \mathbf{h}_{u,i}^{(\ell-1)} + \mathbf{b}^{(\ell)}), \quad \ell = 1, \dots, L, \quad (15)$$

where $\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}$. The final layer output $\mathbf{h}_{u,i}^{(L)}$ is then projected to a scalar score via another affine transformation and sigmoid activation. Bai et al. [12] enhance NCF by incorporating local neighborhood information using convolutional

filters, improving interaction representation between users and items. Liu et al. [106] propose PhD, a hybrid recommendation model combining user-item interactions with side information using probabilistic matrix factorization. Cao et al. [24] develop a group recommendation method integrating an attention mechanism with MLP within the NCF framework. Chen et al. [35] utilize heterogeneous information networks (HIN) to represent diverse data types, capturing complex semantics through meta paths. Fu et al. [53] investigate both local and global co-occurrences in user-user and item-item relations to capture contextual information more comprehensively. Zhou et al. [210] enrich the model's understanding by incorporating textual representations from items, enhancing both explicit and implicit feedback. Chen et al. [39] combine deep feature learning and deep interaction modeling using two MLP networks with a rating matrix. Otunba et al. [122] propose an ensemble framework integrating generalized matrix factorization (GMF) and MLP networks, leveraging the strengths of deep and shallow architectures. Xue et al. [186] focus on capturing high-order item relations in item based CF models using an NCF based approach. Yi et al. [190] develop deep matrix factorization (DMF), integrating diverse side information from users and items to enhance the retrieval process. Chen et al. [31] propose efficient whole-data learning strategies to avoid common sampling techniques in CF retrieval methods. Additionally, closely related methods like CML [68] is based on metric learning, and NBPO [195] employs noisy-label robust learning techniques for retrieval.

4.6 Multi-Interest Methods

The deep neural retrieval methods mentioned above typically assign a single vector to represent each user, encapsulating their preferences and behaviors. However, this single vector representation often falls short in capturing the complexity and diversity of a user's multiple interests. As a result, recent works have developed models that encapsulate multiple interest vectors for each user. Let $\mathcal{H}(u)$ be the set of items with which user u has interacted, and let each item i have an embedding $\mathbf{e}_i$. A general approach is to generate K interest vectors for user u :

$$\{\mathbf{h}_{u,1}, \mathbf{h}_{u,2}, \dots, \mathbf{h}_{u,K}\}, \quad \mathbf{h}_{u,k} = f_k(\{\mathbf{e}_i : i \in \mathcal{H}(u)\}), \quad (16)$$

where each $f_k(\cdot)$ denotes a mechanism, such as dynamic routing or self-attention, that extracts the k^{th} interest representation from the user's historical interactions. Consequently, each $\mathbf{h}_{u,k}$ corresponds to one latent interest of user u . One of the pioneering models is MIND [89], which introduces a dynamic routing mechanism to assign each user interaction to specific interests. ComiRec [27] builds on this by employing a self-attentive mechanism to distill user interests from their activity patterns. Further advancements include models like SINE [149] and Octopus [108], which establish interest pools and use attention mechanisms to activate relevant interests based on historical interactions. DGCF [165] extends these concepts by integrating a dynamic routing mechanism into graph structures, focusing on the independence among interests for enhanced multi-interest learning.

The dynamic nature of user preferences is further explored in works like Pinnersage [123] and MIP [140], emphasizing the importance of models adapting to the evolution of user preferences over time. Models like PIMIRec [32] and UMI [29] integrate time information, interactivity, and user profiles to represent user interests more contextually and temporally. Re4 [199] introduces backward flow regularization to enhance multi-interest learning by capturing the details of user interest evolution. MIRN [204] uses EM routing to segment user behavior for tailored interests. kNN-Embed [48] improves diversity in candidate generation by representing users as mixtures over learned item clusters. HimiRec [124] explores hierarchical interest modeling for structured user preferences. MTMI [181] advocates for a multi-tower architecture to diversify user representation, addressing adaptability and complexity in a dynamic environment.

REMI [178] enhances retrieval quality by mining hard negatives that consider user interests and employing routing regularization for improved robustness and accuracy.

5 INDEXING MECHANISM FOR EFFICIENT RETRIEVAL

After obtaining high-quality user and item representations in Sections 3 and 4, we now discuss efficient indexing mechanisms in modern retrieval systems, which are crucial for organizing and retrieving large-scale items effectively. The indexing methods can be broadly categorized into three main types: (1) no index with whole vault calculations, (2) approximate nearest neighbor search, and (3) tree based indexing.

5.1 Whole Vault Calculation

In the simplest form of retrieval, there is no indexing mechanism, and the system performs whole vault calculations. This approach involves computing the similarity between the user and every item in the candidate pool.

$$s(u, i) = \langle \mathbf{h}_u, \mathbf{h}_i \rangle \quad \forall i \in \mathcal{I}. \quad (17)$$

In equation 17, $\mathbf{h}_u$ and $\mathbf{h}_i$ are the embedding vectors for user u and item i , and $\langle \mathbf{h}_u, \mathbf{h}_i \rangle$ denotes their inner product. While this method can be straightforward to implement, it is computationally expensive and impractical for large-scale systems with millions of items due to the sheer volume of calculations required.

5.2 Approximate Nearest Neighbor Search

To efficiently retrieve the top- K items from a vast candidate item pool after obtaining the embeddings for users and items, approximate nearest neighbor (ANN) search techniques have been developed. These techniques are essential for handling large datasets where full pairwise comparisons would be computationally prohibitive.

$$s(u, i) = g(\mathbf{h}_u, Q(\mathbf{h}_i)). \quad (18)$$

$Q(\mathbf{h}_i)$ represents the index-based, compressed or quantized version of $\mathbf{h}_i$, and $g(\cdot, \cdot)$ is a scoring function that approximates the similarity between $\mathbf{h}_u$ and the indexed representation of $\mathbf{h}_i$. Various techniques and tools facilitate ANN search, such as Faiss [77], Milvus [160], and ScaNN [60], which are instrumental in industry applications for efficiently retrieving the top- K nearest vectors. The construction of popular ANN indices, such as HNSW [114] and IVFPQ in Faiss [77], involves time-consuming procedures, including clustering algorithms like K -means. Despite these challenges, the advantages of using ANN search in practical applications, particularly in speeding up the retrieval process and reducing computational loads, make it a valuable technique in modern retrieval systems.

5.3 Tree Based Indexing

Tree based indexing methods have gained attention in recent years for their ability to efficiently organize and retrieve large-scale items. These methods leverage hierarchical structures to capture the diversity of user interests and represent information more effectively. Tree based indexing methods offer several advantages over vector based approaches. They can significantly reduce the computational complexity by structuring the data hierarchically, enabling logarithmic scale computation relative to the size of the corpus during prediction. Additionally, they allow for the interaction between candidate items and users within the model, enhancing the expressive power of the retrieval method. These methods not only make the use of arbitrary advanced models possible, but also excel at discovering novel and effective

recommendations by exploring the entire dataset and utilizing sophisticated deep models to uncover potential user interests.

TDM [213] is a pioneering work in this area. It ties each item in the corpus to a leaf node via clustering, using a tree structure as an index. To model user preference, TDM assigns to each non-leaf node n at level l a score based on its most promising child at level $l + 1$. Formally, if we denote by $p^{(l)}(n | u)$ the preference of user u for node n at level l , then

$$p^{(l)}(n | u) = \frac{1}{\alpha^{(l)}} \max_{c \in \text{Children}(n)} p^{(l+1)}(c | u), \quad (19)$$

where $p^{(l+1)}(c | u)$ is the preference for child node c in the next finer level, and $\alpha^{(l)}$ is a level-specific normalization constant. In this way, a node's score propagates upward from the highest-scoring child, ensuring that the top- K candidates at level l are drawn from among the children of the top- K nodes at level $l - 1$. The user's overall interest is thus represented by the distribution of scores across the entire tree. At inference time, retrieving the top- K leaf items is equivalent to performing a sequence of small classification tasks—selecting the best nodes level by level—yielding an efficient, scalable retrieval process. JTM [212] improves upon TDM by synchronizing the objectives of index learning and model training, addressing inconsistencies, and enhancing the model's overall performance. OTM [217] further refines this approach by addressing the discrepancy between training and testing in tree based methods. It introduces the notion of bayes optimality under beam search and calibration under beam search, providing general tools for analyzing and improving tree based methods.

Furthermore, Deep Retrieval (DR) [55] innovates by learning a retrievable structure end-to-end, overcoming data scarcity at the leaf level in large-scale recommender systems. DR enables efficient item clustering and retrieval by using a matrix for indexing. These advanced indexing mechanisms play a critical role in improving the efficiency and effectiveness of retrieval stages in recommender systems, offering advantages over early methods by reducing computational loads and enhancing retrieval performance. However, they do require substantial engineering effort to implement, underscoring the need for skilled development to fully realize their performance benefits.

6 OPTIMIZATION METHODS OF RETRIEVAL

In this section, we discuss the learning strategies and optimization methods for the retrieval stage, which involves three main aspects: data preparation, loss functions, and evaluation metrics. In scenarios involving implicit user feedback, the lack of reliable negative data poses a significant challenge. Consequently, data preparation, especially negative sample construction is critical. We will begin with various negative sampling strategies [64, 133] that select a subset of the missing data to act as negative instances, and whole-data based strategies [45, 69, 98] that treat all missing data as negative instances. Then we will explore common loss functions and evaluation metrics used in retrieval tasks.

6.1 Negative Sample Construction

In many real-world scenarios, it is hard to collect explicit user preference. In datasets with implicit feedback, observed interactions indicate a user's positive preference for an item, while all other unobserved items remain unlabeled. This means that only positive feedback is observed, and negative feedback is indistinguishable from missing values in the unobserved data. Additionally, users typically interact with a relatively small selection of items compared to the vast number of items available in real systems. Consequently, the absence of reliable negative data makes learning from implicit feedback particularly challenging. To optimize retrieval methods with implicit feedback, two prevalent strategies have been commonly employed. Negative sampling strategies [64, 133] involve selecting a subset of the missing data to serve as negative instances. While this method is efficient, it can potentially reduce the model's performance due to

its selective nature of sampling. On the other hand, whole-data based strategies [45, 69, 98] treat all missing data as negative instances. It aims to utilize the complete dataset, which can enhance coverage and may improve the model's effectiveness. However, this strategy can be less efficient due to the extensive data processing involved and may lead to the issue of false negatives, where items that are not explicitly interacted with by users are incorrectly treated as irrelevant. We will discuss these strategies in further detail.

6.1.1 Negative Sampling Strategies. The negative sampling strategy extracts negative instances from unobserved data, significantly reducing the training sample size and enhancing the efficiency of the training process [64]. This strategy is prevalent in various retrieval methods, including traditional ones like BPR [133] and neural based ones like NCF [64]. We classify various negative sampling strategies applied in retrieval stage into four types: static negative sampling [65, 193], hard negative sampling [47, 132, 201], adversarial negative sampling [28, 59, 63, 76, 157, 161, 211] and graph based negative sampling [73, 192].

- **Static negative sampling.** When the probability of each item being sampled as a negative instance remains constant throughout training, this sampling strategy is referred to as Static Negative Sampling. Within static negative sampling, the simplest and most widely used method is Random Negative Sampling (RNS), also known as Uniform Negative Sampling. However, this strategy often selects uninformative instances that contribute minimally to model updates, leading to the heuristic strategy of Popularity-biased Negative Sampling (PNS). PNS was first introduced in Word2Vec [119]. [25] has investigated the importance of hyperparameters when applied to recommendation. A more common strategy based on item popularity uses the frequency of an item in the training set as the weight for selecting negative examples, that is, favoring more popular items [65, 193]. This strategy can be explained by popularity bias: if a user does not interact with a highly popular item, it likely indicates disinterest. However, empirical results suggest that PNS does not consistently outperform RNS in retrieval methods [167].
- **Hard negative sampling.** It is challenging for static negative sampling methods to dynamically adapt and adjust the distribution of candidate negative samples. Consequently, they struggle to identify more informative negative samples. Hard negative sampling selects hard negatives that are misclassified or have higher prediction scores, which are more relevant for improving performance [47, 132, 201]. The most common method for mining hard negatives involves selecting samples closest to the user, i.e., those most similar in the embedding space. Other innovative strategies include a sampling-bias-corrected algorithm for more accurate item frequency estimation from streaming data [191] and strategy that utilize both batch and uniformly sampled negatives to mitigate selection bias in implicit recommendations [187].
- **Adversarial negative sampling.** Adversarial negative sampling methods typically involve a generator and a discriminator. The generator acts as a sampler, creating samples to confuse the discriminator, which must determine whether a given sample is a positive instance or one generated by the generator. The goal is still to learn a better distribution of negative samples. Some GAN based methods adaptively evolve the sampling probability by optimizing adversarial objectives [28, 59, 63, 76, 157, 161, 211]. However, adversarial negative sampling faces significant challenges: its complex framework, unstable performance, and lengthy training times all limit its practical applications. Additionally, the competition between the generator and discriminator may not always converge to an ideal Nash equilibrium, indicating that there is still room for exploration and improvement.
- **Graph based negative sampling.** Hard negative sampling and adversarial negative sampling effectively utilize the semantic information in the embedding space, while graph based negative sampling methods further integrate the structural information of samples within the graph. For instance, [192] sampled negative nodes based on their

PageRank scores. [73] proposed MixGCF, which implemented two strategies: positive mixing and hop mixing. Positive mixing enriches original negative samples with positive embeddings to infuse them with positive representations, while hop mixing enhances negative samples through GNN aggregation of neighborhood information. These strategies have achieved SOTA results in sampling methods for GNN based recommender systems.

Additionally, there are some cutting-edge efforts in the field. For instance, while negative sampling strategies primarily aim to improve the quality of negative samples, the ratio of negative sampling determines the quantity of these samples. SimpleX [116] has shown that even basic CF methods, when combined with the appropriate negative sampling ratio and loss function, can outperform current SOTA retrieval methods. Additionally, in scenarios where only positive and unlabeled data are accessible, sampling inevitably introduces certain biases, such as the false negative issue, a typical example of sample bias. CLRec [209] theoretically demonstrates that popularity based selection in contrastive loss equates to reducing exposure bias through inverse propensity weighting, offering a new perspective on understanding the effectiveness of contrastive learning.

6.1.2 Whole-Data Based Strategies. Whole-data based strategies, also known as non-sampling strategies, treat all unobserved data as negative examples but assign them a lower weight than positive examples. This approach ensures that the entire dataset is utilized, potentially offering better coverage. For instance, WMF [69] assigns a uniform weight to all unobserved entries. EALS [65] and ExpoMF [98] adjust the weight of unobserved entries based on item popularity. The rationale is that popular items are more likely to be seen by users and should therefore be given higher weights as negatives. These methods have demonstrated the ability to leverage whole datasets with potentially better coverage [65, 98, 179], while inefficiency is the challenge.

To mitigate inefficiency, several methods have been developed, such as batch based ALS [16, 65, 69] and mini-batch SGD [179]. These techniques accelerate the learning process, making whole-data based strategies more practical. However, these methods are generally suitable only for traditional retrieval methods with a linear prediction layer [16, 31, 179] and regression loss function. Consequently, non-sampling methods have been widely applied in traditional retrieval methods [65, 69, 98] but are less common in neural retrieval methods. Recently, the neural non-sampling retrieval method ENMF [31] was proposed, representing an effort to apply non-sampling strategies within neural frameworks. Individuals who are interested in this part can refer to [30] for further details.

6.2 Loss Functions

In this part, several commonly used loss functions in retrieval methods will be introduced.

6.2.1 Sampled Softmax Loss. In recommender systems, calculating the softmax over a vast number of items is computationally expensive, making it impractical for real-time retrieval. By approximating the softmax function through negative sampling, sampled softmax loss is often used to address the computational inefficiency of the softmax function when dealing with large-scale item sets in retrieval stage. As is shown in equation 20, sampled softmax loss approximates the denominator by considering only a subset of sampled negative items.

$$L_{\text{SampledSoftmax}} = -\frac{1}{|B|} \sum_{(u_i, v_i) \in B} \log \frac{\exp(G(u_i, v_i))}{\exp(G(u_i, v_i)) + \sum_{j \in S_i} \exp(G(u_i, v_j))} \quad (20)$$

$$G(u, v) = \mathbf{u} \cdot \mathbf{v} - \log(Q(v|u)) \quad (21)$$

B stands for the mini-batch and u_i, v_i represent the embedding of user i and item i , respectively. $G(u_i, v_i)$ is the scoring function that calculates the interaction score between user u_i and item v_i . S_i refers to the set of sampled negative items

for user u_i . These are items that the user has not interacted with and are considered as negative examples during training. Usually, we compute the matching score between user and item using dot product and adjust based on the sampling probability $\log(Q(v|u))$ to prevent popular items from being overly penalized. This adjustment occurs only during training; for predictions, we rely solely on the dot product, which is compatible with vector databases like Faiss [77] and Milvus [160].

Sampled softmax loss ensures that the model focuses on differentiating between the positive items and the selected negative samples, and is particularly useful in the retrieval stage of recommender systems, where the goal is to quickly and accurately identify a subset of relevant items from a large item pool. By reducing the computational overhead, it enables faster training and inference times, making it feasible to deploy in real-world scenarios with millions of items.

6.2.2 Noise Contrastive Estimation Loss. Noise Contrastive Estimation(NCE) loss is an efficient training objective for large-scale recommender systems, particularly in the retrieval stage. It addresses the challenge of computing probabilities over an extensive set of items by transforming the problem into a binary classification task. NCE approximates the softmax function by contrasting observed data (positive examples) against artificially generated noise (negative examples), simplifying the likelihood computation. NCE loss can be formulated as equation 22, where B stands for the mini-batch, and $G(u_i, v_i)$ is the matching score between user u_i and item v_i in equation 21. σ is the sigmoid function and S_i represents the set of negative items sampled for user u_i .

$$\mathcal{L}_{\text{NCE}} = -\frac{1}{|B|} \sum_{(u_i, v_i) \in B} \left[\log \sigma(G(u_i, v_i)) + \sum_{j \in S_i} \log(1 - \sigma(G(u_i, v_j))) \right] \quad (22)$$

During training, the method learns to differentiate between true user-item interactions and sampled noise. By adjusting the parameters to maximize the likelihood of observed interactions and minimize the likelihood of noise, NCE effectively handles the computational complexity of large-scale retrieval tasks.

6.2.3 Pairwise Loss. Pairwise loss is a type of Learning-to-Rank approach commonly used in retrieval methods. It focuses on optimizing the ranking of items for a given user by comparing pairs of items, which ensures that preferred items are ranked higher than less preferred ones. One common method to implement pairwise loss is using Marginal Hinge loss, defined as:

$$L_{\text{Hinge}} = \frac{1}{|B|} \sum_{(u_i, v_{i+}, v_{i-}) \in B} \max(0, m - \mathbf{u}_i \cdot \mathbf{v}_{i+} + \mathbf{u}_i \cdot \mathbf{v}_{i-}) \quad (23)$$

where B is a batch containing triplets (u_i, v_{i+}, v_{i-}) . $\mathbf{u}_i$ is the embedding vector for user u_i and $\mathbf{v}_{i+}$ and $\mathbf{v}_{i-}$ are the embedding vectors for the positive and negative items, respectively. m is a margin parameter ensuring a minimum difference between the scores of positive and negative items. The goal is to ensure the matching degree of positive items is higher than that of negative items by at least the margin m .

Bayesian Personalized Ranking (BPR) loss is another pairwise loss, which maximizes the probability that a positive item is ranked higher than a negative item. It is defined as:

$$L_{\text{BPR}} = -\frac{1}{|B|} \sum_{(u_i, v_{i+}, v_{i-}) \in B} \log \sigma(\mathbf{u}_i \cdot (\mathbf{v}_{i+} - \mathbf{v}_{i-})), \quad (24)$$

where σ is the sigmoid function. This formulation avoids the need for a margin parameter and focuses on maximizing the ranking order directly.

7 EVALUATION METRICS

In this section, we collect and summarize the most commonly used metrics for assessing retrieval-stage performance. We split these into (1) offline metrics, which are computed on held-out logs or public datasets, and (2) online (A/B) metrics, which are measured in a live serving environment.

7.1 Offline Metrics

Offline evaluation is typically performed on historical logs or public benchmarks. These metrics measure how well a retrieval method ranks known positive interactions versus held-out negatives or unobserved items. Recall@K, Precision@K, and F1@K are closely related metrics in the retrieval stage. We categorize the ground truth labels as either positive or negative, within the predicted labels, this results in classifications such as True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). Recall and Precision indicate the proportion of correctly predicted positive samples relative to the predicted positives and ground truth positives, respectively. The F1-score combines recall and precision into a single metric, as is shown in equation 25.

$$Recall = \frac{TP}{TP + FN} \quad Precision = \frac{TP}{TP + FP} \quad F1 = \frac{2 * precision * recall}{precision + recall} \quad (25)$$

Hit Rate(HR) is another popular metric, which measures the proportion of hit items in the predicted top- K list relative to the total items in the test set. A hit occurs if a user interacts with any of the top- K retrieved items, which can be formulated as:

$$HR@K = \frac{NumberofHits@K}{Test}. \quad (26)$$

While Precision and Recall do not account for the order of items, Average Precision(AP) calculates the average precision score across multiple items within the list. It considers the precision at each rank and takes into account both the presence and position of relevant items, giving higher scores to relevant items appearing earlier in the list. Mean Average Precision (MAP) extends AP by averaging the AP scores across all users. AP and MAP are formulated in equation 27, where $rel(k)$ indicates relevance and p_k is the precision at cut-off k .

$$AP@K = \frac{1}{|K|} \sum_{k=1}^K \frac{p_k \cdot rel(k)}{k} \quad MAP = \frac{1}{|U|} \sum_{u \in U} AP_u \quad (27)$$

7.2 Online Metrics

Online evaluation is conducted via A/B testing on live traffic. The primary metrics include click-through rate (CTR), conversion rate (CVR), revenue per mille (RPM), and gross merchandise value (GMV), which are defined in equation 28, where # denotes the quantity:

$$CTR = \frac{\#clicks}{\#impressions} \quad CVR = \frac{\#conversions}{\#clicks} \quad RPM = \frac{Revenue}{\#impressions} \times 1000 \quad GMV = \sum_{i \in S} price_i \times quantity_i. \quad (28)$$

When measuring CTR, an ‘impression’ is recorded when an item is rendered in a user’s viewport for at least one second, and a ‘click’ is recorded when the user taps or clicks that item. Closely related to CTR is CVR, which measures the proportion of clicks that result in a downstream action (e.g., ‘Add-to-Cart’ or ‘Purchase’). Since CVR reflects actual revenue-generating behavior, it typically serves as the primary business key performance indicator once engagement (CTR) reaches satisfactory levels. RPM indicates revenue generated per thousand impressions. GMV represents the total monetary value of all transactions originating from recommended items, where S denotes the set of items purchased.

Table 1. Statistics of the datasets.

Dataset	# Users	# Items	# Samples	Density
MovieLens-1M	6,040	3,706	1,000,209	0.04468
Amazon-Book	52,643	91,599	2,894,108	0.00062
Yelp2018	31,668	38,048	1,561,406	0.00130

8 DATASETS AND EXPERIMENTS

In this section, we first introduce commonly used large-scale public datasets for retrieval tasks and select representative retrieval methods to evaluate their performance across three datasets. Additionally, since the study of retrieval methods is deeply rooted in practical applications and faces numerous real-world challenges, designs purely based on theoretical research may not fully capture the potential benefits of innovations at the retrieval stage. Therefore, we summarize online performance statistics from published papers to gain further insights.

8.1 Datasets

Many real-world datasets have been released to advance research in retrieval methods, such as MovieLens [7], Yelp [4], AmazonBooks [3], Gowalla [6], and Criteo [5]. Additionally, a large collection of preprocessed datasets is available in RecBole [205]. However, most of these datasets are relatively small, typically containing fewer than millions of interactions between tens of thousands of users and items, which differs significantly from the large-scale data in industry. Therefore, we will primarily introduce some larger, industrial-scale datasets.

- **MovieLens** [7] dataset is a widely used dataset in the field of recommender systems. It was released by the GroupLens Research Project at the University of Minnesota. The dataset contains user ratings for movies, along with some additional information such as user age, occupation, zipcode and item genre. MovieLens-1M and MovieLens-20M are commonly used in various research studies [62, 116].
- **Gowalla** [6] dataset was collected from the Gowalla social network, which was a location based social networking website where users could check in at physical locations and share their activities with friends.
- **Amazon-Book** [3] dataset is a sub-dataset from the Amazon review dataset, which contains millions of reviews written by Amazon customers for various products available on the Amazon e-commerce platform. Additionally, other sub-datasets in Amazon, such as Amazon-CDs, Amazon-Movies, Amazon-Beauty, and Amazon-Electronics, are also used in some research studies [116, 117, 143].
- **Yelp** [4] dataset consists of reviews and related information collected from the Yelp platform, which is known for providing crowd-sourced reviews about businesses, particularly restaurants and other local services.
- **CiteULike-A** [8] dataset was gathered from CiteULike and Google Scholar. CiteULike enables users to create their own collections of articles. Each article includes its abstract, title, and tags.
- **MillionSongData** [9] dataset is a freely available collection of audio features and metadata for a million contemporary popular music tracks. Typically, play counts are binarized and interpreted as implicit preference data.

To provide a clear understanding of the performance of various retrieval methods, we present the results of several retrieval methods on the MovieLens-1M, Amazon-Book, and Yelp2018 datasets below. For Amazon-Book and Yelp2018, following previous work [61, 64], we apply the 10-core filtering to ensure data quality, retaining only users and items with at least ten interactions. Due to its relatively smaller size, MovieLens-1M is used in its original form without core filtering. The statistics of these datasets are shown in Table 1.

Table 2. The best result is given in bold, and the second-best result is underlined. All the benchmark models are trained under the same setting and the experiments have been run for three times to get the average metric results.

Methods	MovieLens-1M		Amazon-Book		Yelp2018	
	HitRate@50	Recall@50	HitRate@100	Recall@100	HitRate@100	Recall@100
Random	0.1819	0.0138	0.0062	0.0012	0.0119	0.0027
Pop	0.7703	0.2058	0.1072	0.0305	0.1561	0.0490
ItemKNN [137]	0.9191	0.3896	0.6397	0.2996	0.5792	0.2488
BPR [133]	0.9322	0.4198	0.5117	0.2028	0.5360	0.2148
ENMF [31]	0.9435	0.4419	0.5593	0.2348	<u>0.6129</u>	0.2644
NGCF [164]	0.9369	0.4256	0.5062	0.1980	0.5834	0.2455
DGCF [165]	0.9399	0.4326	0.5334	0.2165	0.5878	0.2498
LightGCN [62]	0.9412	0.4334	0.5574	0.2317	0.5927	0.2543
SGL [172]	0.9422	0.4320	0.5882	0.2527	0.6264	0.2771
CDAE [174]	0.9482	0.4407	0.5307	0.2189	0.5807	0.2463
MultVAE [99]	0.9505	<u>0.4497</u>	0.5369	0.2222	0.5937	0.2566
MultDAE [99]	<u>0.9508</u>	0.4462	0.5596	0.2368	0.6085	0.2655
RecVAE [138]	0.9551	0.4683	<u>0.5627</u>	<u>0.2403</u>	0.6094	<u>0.2680</u>

8.2 Experiment Settings

Evaluation Metrics. We adopt two commonly used metrics to evaluate the performance of various retrieval methods: HitRate@K and Recall@K. Specifically, we set K=50 on MovieLens-1M, and K=100 on Amazon-Book and Yelp2018 datasets. See Section 7 to have a deeper understanding of different metrics.

Methods. We comprehensively evaluate 11 well-known retrieval methods including CF based, GNN based and autoencoder based. We also implement random and popularity based retrieval for comparison. Detailed results can be found at Table 2.

Implementation Details. As with previous work [64], for each dataset, we randomly select 80% of each user’s historical interactions for the training set and use the remaining 20% as the test set. Additionally, we randomly select 10% of the interactions from the training set to serve as the validation set. We implement these methods using the RecBole open-source framework¹ and its provided code [205], and we adopt the optimal hyperparameters listed on the RecBole website.²

8.3 Experimental Results

In Table 2, we showcase a variety of contemporary retrieval methods, highlighting key examples from categories such as CF based, GNN based, and autoencoder based approaches. The top-performing methods are indicated in bold, while the second-best results are underlined, offering multiple insights.

First, there has been a continuous evolution within each category, leading to enhanced performance through more precise and tailored modeling techniques. Second, it appears no single model type uniformly excels over others across all datasets. For example, RecVAE [138] leads the field on the Movielens-1M dataset, ItemKNN [137] stands out on Amazon-Book, and SGL [172] excels on Yelp2018. This diversity in leading models suggests that the field of retrieval remains wide open for innovation and is not dominated by any particular approach, indicating significant potential for

¹<https://github.com/RUCAIBox/RecBole>

²<https://recbole.io/hyperparameters/index.html>

Table 3. Industrial online deployments of various retrieval methods claimed in published papers.

Model Class	Model	Compared to	Performance Improvement
Traditional Methods	Swing [188]	Item-Based CF	+9.3% CTR, +17.6% CVR, +20.3% PPM
	EGES [159]	Item-Based CF	+1.30% CTR
Graph-Based Methods	PinSage [192]	Methods Employed before	+10% ~30% Repin Rate
	NIA-GCN [143]	DSSM [72]	+10.19% CTR, +9.95% CVR
Two-Tower Methods	YoutubeDNN[191]	Methods Employed before	+0.37% Engagement
	MNS[187]	Sampled Softmax instead of Mixed Negative Sampling	+1.54% High-Quality App Install Gain
	DAT[196]	Basic Two-Tower Method	+4.17% CTR, +3.46% GMV
	MVKE[185]	Basic Two-Tower Method	+0.79% GMV, +0.51% Adjust Cost
Multi-Interest Methods	MIND[89]	Item-Based CF	+2.85% CTR
	PinnerSage[123]	Single User Embedding	Homefeed Engagement: +4% Volume, +2% Propensity Shopping Engagement: +20% Volume, +8% Propensity
	UMI[29]	Methods Employed before	+0.95% pCTR, +8.84% GMV, +5.35% ANP +1.37% NAC ¹ , +1.79% Diversity
	HimiRec[124]	ComiRec [27]	+0.31% User Activity, +0.76% Click, +2.00% Engagement +0.45% Retention, +1.26% Diversity
Advanced Indexing Schemes	TDM[213]	Methods Employed before	+2.1% CTR, +6.4% RPM
	JTM[212]	Item-Based CF	+11.3% CTR, +12.9% RPM
	DR[55]	Methods Employed before	+3.0% Video Finish Rate, +0.87% App View Time + 0.036% Second Day Retention
Methods on Cascade System	ARF[170]	LambdaLoss [23]	+1.5% Revenue, +2.3% Conversion
	FS-LR[206]	Methods Employed before	+0.12% Usage time, +0.50% Real show, +0.69% Click +0.40% Like, +0.74% Follow, +0.94% Forward, +1.08% Comment, +0.18% Watch Time

¹ PPM: Payment Per Thousand Impressions; RPM: Revenue Per Mille; ANP: Averaged Number of Payments; NAC: Number of Adding to Cart; Repin Rate: Percentage of Homefeed Recommendations saved by users in Pinterest; Adjust Cost: The Cost paid by advertisers to the platform, Adjusted based on actual costs.

breakthroughs across various categories. For an in-depth analysis of performance of various models, please refer to BARS benchmark [215].

8.4 Industrial Practices

The study of retrieval methods is deeply rooted in practical applications and faces numerous real-world challenges. Therefore, designs purely based on theoretical research may not fully capture the potential benefits of innovations at the retrieval stage. In this section, we explore the industrial applications of key retrieval methods. To guarantee the reliability and accuracy, we only include online performance statistics from published papers. The results are summarized in Table 3, which leads to the following observations.

Firstly, the retrieval stage often deals with large-scale item sets, making experiments on industrial online platforms particularly crucial. Examples include YouTube’s YouTubeDNN [191], Airbnb’s personalized embeddings [57], Pinterest’s PinSage [192], and Taobao’s EGES [159] and TDM [213], to name a few. These applications span a wide range of scenarios, including online advertising, e-commerce, and online video streaming, underscoring the pivotal role of retrieval in recommender systems. Moreover, recent industrial practices in retrieval have shown a trend toward innovative approaches, such as enhancements to dual-tower structures, precise capturing and modeling of users’ multiple interests, and more cutting-edge techniques like advanced indexing schemes and studies on cascade systems, rather than relying solely on a single retrieval stage. These developments are tailored to fit the unique needs of industrial recommendation scenarios. We will explore these future directions in greater detail in Section 10.

9 CASE STUDY OF CURRENT INDUSTRIAL PRACTICES

In this section, we present a detailed case study of the retrieval pipeline at China Merchants Bank (CMB), a mainstream bank company. We focus on CMB Life APP for credit card³, which attracts millions of daily active users and generates billions of user logs each day through implicit feedback, such as click behaviors. CMB Life APP has enriched its online service ecosystem by integrating various scenarios. In the meal-and-movie ticketing service, users receive recommendations for merchants offering personalized dining and cinema discounts. In shopping malls, consumers are presented with products redeemable through accumulated loyalty points. On homepage feeds, trending news items appear based on users' interests. In automotive marketplaces, platforms recommend car brands alongside customized down-payment and monthly-payment plans tailored to individual financial profiles. Each scenario serves different business units with distinct performance metrics, such as click-through rates and conversion rates.

9.1 The Overall Pipeline

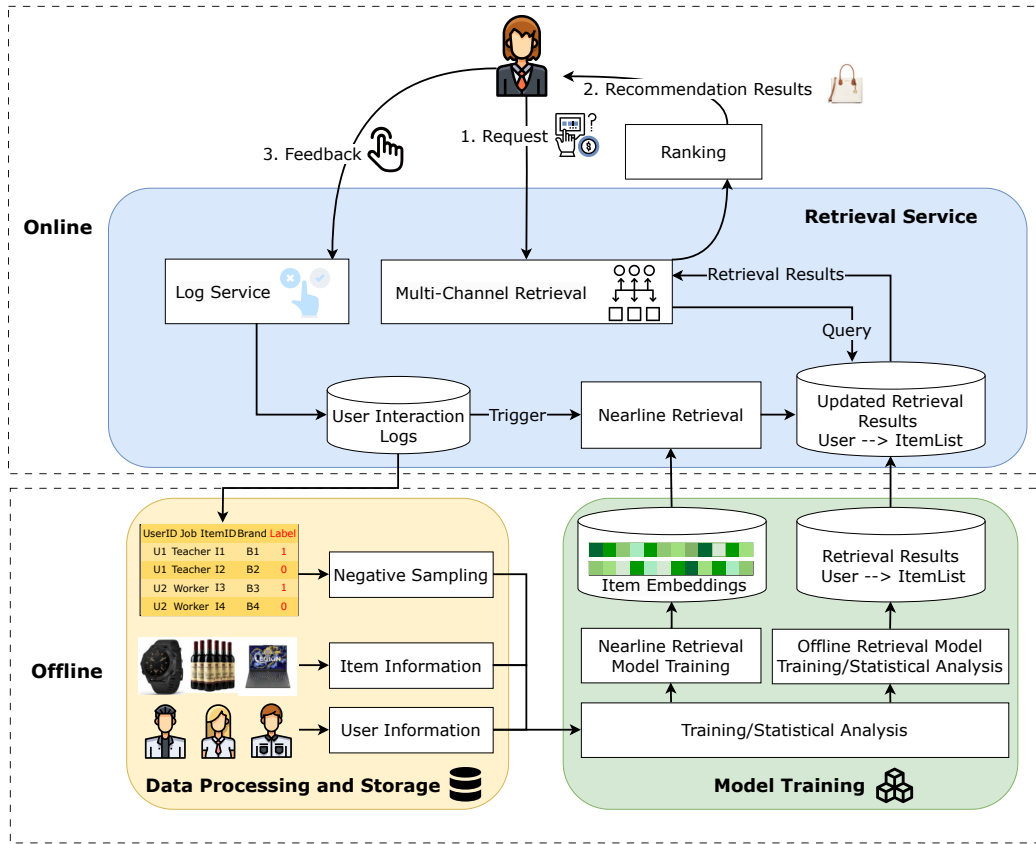


Fig. 6. The retrieval pipeline in CMB Life APP for credit card.

³<https://english.cmbchina.com/CreditCard/>

We outline the overall recommendation pipeline at CMB Life APP for credit card in Figure 6. The pipeline is broadly divided into two main parts: offline training (with data storage), and the online recommendation service. Typically, offline processes are responsible for storing item embeddings and retrieved item list for each user. When a user makes an online request, the retrieval stage queries the k most similar items from the stored retrieval results and forwards them to the subsequent stages including ranking and re-ranking, culminating in the display of recommendation results to the user. Users' interactions with these recommendation results, such as clicks or purchases, are logged as user interaction logs, which serve as data for offline training of retrieval models. This whole process forms a continuous feedback loop. We will next provide in-depth explanations of data processing and storage, training, and recommendation services, respectively.

In the data processing and storage section, we utilize three types of data sources. (1) *Item information*: The item service provides detailed information about items, such as titles, prices, e.t.c. (2) *User information*: This primarily includes basic user profiles. (3) *User interaction logs recorded by various business systems*: These logs capture user-item interactions such as views, clicks, purchases, and favorites. As mentioned in Section 6.1.1, due to the need to score a large number of items that have not been previously exposed, relying solely on ranking data with user feedback can lead to *sample selection bias* (SSB) [33], which is essentially a mismatch between training and inference data. This is akin to forcing a student to take a test on material not covered in the syllabus. To address this, negative sampling is needed. In the specific context of Mobile Life, we utilize random negative sampling, whose proportion depends on the volume of data and the balance of positive and negative samples. These three data sources are combined and used for offline training or statistical analysis of retrieval models.

As for retrieval models, we categorize them into two types: offline retrieval and nearline retrieval.

- **Offline retrieval.** Offline retrieval typically involves methods that do not require real-time updating, such as popular or new item retrieval. Once offline statistical analysis is complete or training process is finished, the retrieval results, such as user-to-item lists, are generated and saved for quick access during retrieval.
- **Nearline retrieval.** Nearline retrieval involves methods that require periodic training and updating, which necessitates maintaining item embeddings. When a user's interaction history updates, nearline retrieval is triggered. To be specific, if a user recently clicked on an item, nearline retrieval select the top items from the item pool based on the dot product of item embeddings. The retrieved items are then combined with the offline user-to-item list to create an updated retrieval list.

During online recommendation service, when a user initiates a request, the system leverages multi-channel retrieval to query the user-specific retrieval pool, combining both offline and nearline results. These results are then passed to subsequent ranking models, ultimately displaying the final recommendations to the user.

9.2 Multi-Channel Retrieval

As illustrated in Figure 7, our industrial setting features seven retrieval channels, which includes three nearline I2I retrieval methods, one offline U2I retrieval method, and three other statistical based retrieval methods. During online service, the I2I approach retrieves the top- K similar items using the embedding of an item that a user clicks or purchases and places them into the user's retrieval pool; the U2I retrieval model retrieves directly through the user ID; and the statistical based retrieval methods fetch the top- K items directly from the database based on predefined quotas. We will now introduce each channel.

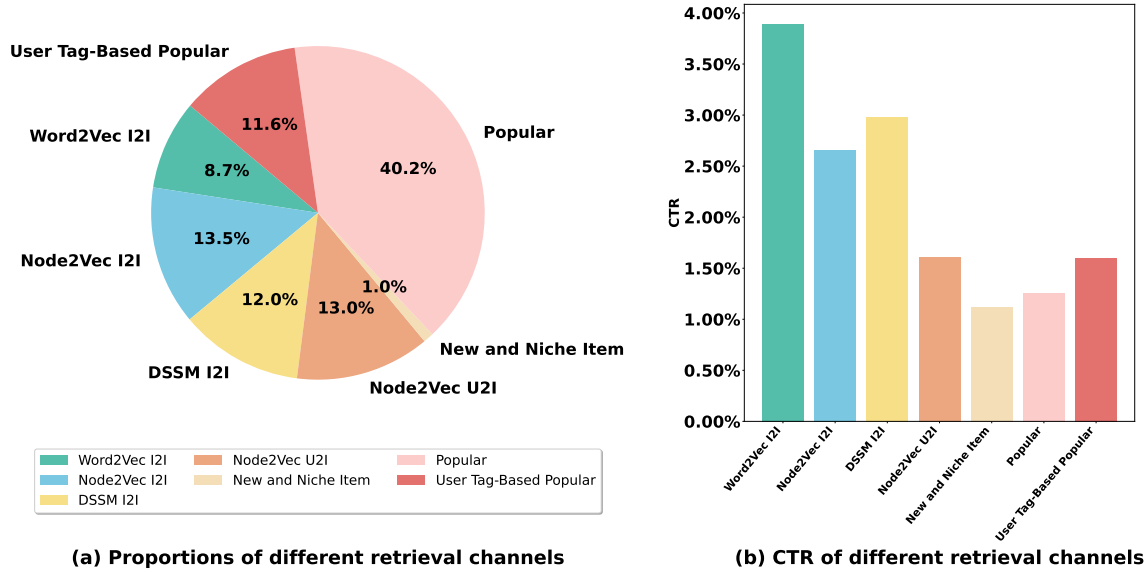


Fig. 7. Overview of multiple channels in the retrieval stage of CMB Life APP for credit card.

For Word2Vec, we feed comprehensive text fields from within the scenario such as item titles, attributes, and categories to generate item embeddings, which are then stored in an embedding database. The underlying principle is that items with similar titles, attributes, and categories are more likely to appeal to users. For Node2Vec, we create positive sample pairs from user behavior sequences by using a sliding window of length 2 to generate positive item pair item i and item j . Simultaneously, we form negative item pairs by pairing item i with a random item k from the item pool, ensuring that $k \neq j$, to train the Node2Vec retrieval model. In the setting of I2I, we directly store the generated item embeddings, while in the setting of U2I, we obtain user embeddings by weighting the embeddings of the recent sequence of items interacted with by the user. We then use the user embedding to retrieve the top- K items, storing them in the format of User→ItemList. The rationale is that items interacted with consecutively in a user's behavior sequence are perceived as more similar by the user, making it likely that they will engage with other items close to item i after visiting it. For DSSM, we generate positive user-item pairs from user behavior sequences and create negative samples by pairing the user with a randomly sampled item from the item pool. These data are then used to train the DSSM retrieval model, which yields item embeddings that are then stored in the embedding database. DSSM may offer more personalization than Node2Vec in this scenario, as the implicit graph structure includes both users and items.

Additionally, we have three retrieval methods based on statistical analysis. The New and Niche Item retrieval refers to storing recently least exposed items, based on the idea that new and niche items typically have lower exposure and interactions, leading to significant bias in interaction volume predictions. Exposing a certain proportion of such items can unearth overlooked items. The Popular retrieval method involves scoring items based on recent clicks, exposures, favorites, and purchases, then storing those with the highest scores. This method is especially useful for supplementing retrieval results for some users, particularly inactive or new users, by providing popular items as a fallback. User Tag Based Popular retrieval categorizes users into buckets based on fundamental attributes, which may yield more relevant results.

Figure 7 displays the distribution and online performance of the seven retrieval channels mentioned above. It is important to note that this section evaluates the live production system, diverging from the offline benchmark analysis in Section 8.3; as the experimental contexts and objectives differ, the results from these two sections are not directly comparable. In the retrieval stage, the focus is on covering potential user interests with various approaches, making the integration of multiple retrieval results a significant topic. Currently, in the scenario of CMB Life APP for credit cards, our retrieval strategy is divided into two layers. The first layer includes multiple retrieval methods, consisting of three I2I retrieval methods, one U2I retrieval method, and a retrieval method for new and niche items. The second layer involves supplementary retrieval approaches, including popular item retrieval and user tag based popular item retrieval. This supplementary layer is only activated if the first layer's results are insufficient. For example, if we aim to retrieve 300 items and the first layer retrieves only 180, the second layer will add the top 120 to reach 300. If the first layer already retrieves 300 items, the second layer will not be triggered. As seen in Figure 7, supplementary retrieval accounts for about 50% in this scenario, largely due to the presence of many inactive users.

9.3 Key Insights and Challenges

In this section, we discuss some insights and challenges during the efforts in the retrieval stage.

- During the training of retrieval models, attention must be paid to the construction of negative samples and the sampling ratios. The proportion of negative sampling largely depends on the volume of data and the balance of positive and negative samples. Typically, the optimal sampling ratio is determined by evaluating offline metrics. Different sampling ratios can be tested to identify the best performing ratio, similar to a hyperparameter grid search. This careful tuning ensures that the model effectively distinguishes between relevant and irrelevant items, thereby improving the overall performance of retrieval.
- As retrieval methods are followed by ranking and re-ranking stage and do not directly target final business metrics like CTR and CVR, there can be discrepancies between online and offline metrics. The primary reason for this inconsistency is that the retrieval stage is just the first part of the cascade system. Subsequent stages, such as ranking and re-ranking, further filter and prioritize the retrieved items based on more complex models. Therefore, offline evaluations might not fully capture the impact of these later stages. As a result, online experiments are often necessary to assess the actual performance of various retrieval methods, as they provide a more holistic view of the end-to-end recommender system's effectiveness.
- Multi-channel retrieval is essential for maximizing recall rate. By using different retrieval strategies, we can cover a wide range of user preferences, thereby improving overall recall. Even if channels focusing on new and niche items have a slightly lower CTR, they still contribute significantly to enhancing the overall recall rate.
- In the context of the CMB Life APP for credit cards, the integration strategy employs a multi-channel retrieval approach with additional supplementary retrieval methods. This involves how to adjust the volume of retrieved items from each channel. Also, snake-merge is commonly used in industry, which involves alternating the selection from each retrieval channel to create a mixed list of retrieval results. However, these methods are quite rudimentary, and needs better ways to adjust the weights and integrate retrieval results from multiple retrieval channels.

10 CHALLENGES AND FUTURE DIRECTIONS

In this section, we propose several open challenges and potential future directions related to retrieval methods in recommender systems. Some of these topics are crucial yet underexplored in the field, while others present promising avenues for future research.

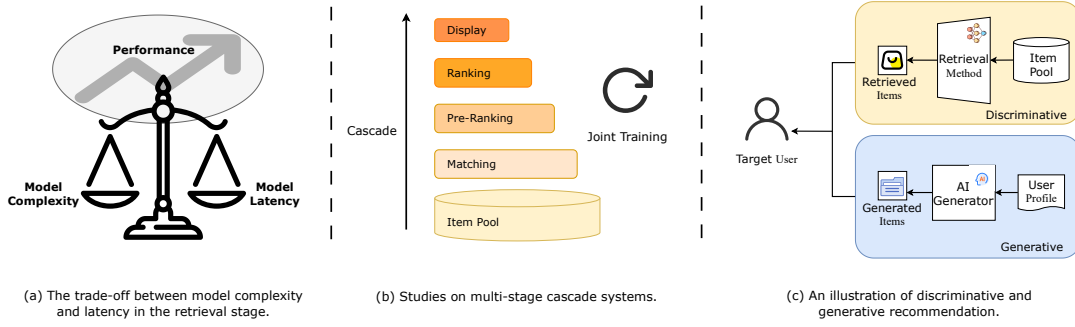


Fig. 8. A depiction of several underexplored challenges and possible future directions.

10.1 From Coarse Features to More Informative Interaction Patterns

As mentioned in Section 1, retrieval differs from ranking. Due to the large-scale candidate item set and efficiency constraints at the retrieval stage, it is challenging to use complex cross-features in retrieval methods. Consequently, many earlier approaches, such as the commonly used two-tower methods [43, 72], separately obtain user and item representations and then compute a simple dot product or cosine similarity. However, these coarse features of users and items fail to provide comprehensive representations. Therefore, recent work has focused on balancing and making trade-offs between performance, model complexity, and real-time latency.

- **Enhance the depth of interaction between the user and item side.** For instance, DAT [196] uses an adaptive-mic mechanism to implicitly model interactions between the two towers. Another approach, IntTower [92], enhances the original two-tower methods by interacting the last hidden vector on the item side with multiple hidden vectors on the user side explicitly. Additionally, it employs positive and negative sample pairs to guide the model through self-supervised learning, capturing interactive signals implicitly. MVKE [185] generates multiple user interest representations, which are then aggregated through an attention mechanism with the item representations to refine each user's item-related representations.
- **Knowledge distillation.** Moreover, models like PFD [182] have been developed where two-tower and single-tower models are trained in parallel. The rich information of the single-tower model guides the learning of the two-tower model, with only the two-tower model being used for online retrieval services after training. During offline training, the teacher model includes a large number of informative features, such as interaction features that the two-tower methods cannot access, like clicks within the same category by a user in the past 24 hours, and post-click information that is unavailable during online services. The teacher model does not require pre-training and is updated synchronously with the student model. Initially, both models update independently until the teacher model stabilizes, at which point distillation begins. Beyond recommendation, some research on knowledge distillation has also been applied to information retrieval (IR) tasks, such as ENDX [168], TRMD [41], VIRT [90], ERNIE-Search [111].
- **Advanced indexing schemes.** In the retrieval stage, efficient indexing schemes are essential for managing and accessing large-scale items effectively. Tree based methods [213], while offering enhanced expressive power and the ability to capture diverse user interests through hierarchical representations, also present significant challenges. The primary drawback is the need for tight integration between algorithm and engineering, requiring customized indexing structures that demand substantial engineering effort. Unlike two-tower methods that can leverage open-source tools

like Faiss [77], tree based methods necessitate bespoke solutions, complicating deployment and optimization. The co-design approach of aligning model and index, although addressing consistency issues, adds further intricacy.

10.2 From Single-Stage Retrieval Optimization to Cascade System Studies

In recent years, there has been a notable shift from single-stage retrieval models to exploring multi-stage cascade ranking systems [70, 74, 170, 214]. For instance, Gallagher et al. [54] explore deriving gradients for cascade rankers, employing end-to-end methods for optimization, while Fan et al. [49] advocate for unifying multiple stages into a single stage using hard negative sampling. Furthermore, Fei et al. [50] suggest a strategy to share features across different stages. Hron et al. [67] suggest using multiple channels together during the retrieval stage and learning how to merge the items retrieved from various channels. Rankflow [129] suggest training rankers on their specific data distributions to alleviate the SSB issue and better utilize the interactions between each stage. In CoRR [74], the retriever is enhanced by deriving high-quality training signals from the ranker, while the ranker benefits from learning to discriminate hard negatives sampled by the retriever. GPRP [206] addresses both the selection bias in each stage of the system pipeline and the underlying interests of users. The core idea is to estimate the selection bias in subsequent stages and then develop a ranking model that aligns with this bias, ensuring the top-ranked results are delivered in the final output. RAR [70] stands for Recall Augmented Ranking, which optimizes the recommendation process by utilizing multi-stage information. ARF [170] introduces a novel perspective on optimizing cascade ranking systems by emphasizing the adaptability of optimization targets to data complexities and model capabilities. These studies highlight the importance of viewing the retrieval process from a cascade system perspective, where each stage is optimized not in isolation but in concert with the others to improve the overall performance and effectiveness of the recommender system.

10.3 From Discriminative Recommendation to Generative Recommendation

Discriminative recommendation retrieve items from an existing corpus via scoring. However, this retrieval based approach has two main drawbacks: (1) the predefined items in the corpus may not meet the diverse information needs of all users, and (2) users typically provide feedback through passive interactions like clicks, which is inefficient and often imprecise. Recent advancements in AI-Generated Content (AIGC) offer promising solutions to these issues. Generative AI can create personalized items that better align with specific user needs. Moreover, large language models (LLMs) with advanced language understanding and generation capabilities make it easier for users to express their preferences through natural language, reducing the effort required to tailor recommendations [101]. Generative recommendation differ significantly from discriminative recommendation. Unlike discriminative recommendation, which typically rely on pointwise scoring, generative models streamline the process by directly generating recommendations.

In rating prediction tasks, several studies have tested LLMs, many of which are based on ChatGPT [44, 105, 169, 176, 208]. Recent studies have found that the context length limit of an LLM makes it impractical to input all items. To address this, two approaches have been explored. The first is straightforward recommendation, which uses a prompt containing only user information (e.g., ID or metadata) and asks the LLM to generate recommendations directly [46, 184, 197]. The second approach is selective recommendation, which includes both user information and a list of candidate items in the prompt, asking the LLM to select items from these candidates [44, 56, 91, 95, 198].

Some recent studies [94] have also instructed LLMs to determine whether a user will interact with a given item by generating ‘yes’ or ‘no’ responses [13, 103]. Although these responses are generated by LLMs, this approach is still considered discriminative recommendation since it generates an answer or a probability score for each item. The shift

towards generative recommendation highlights a new paradigm in recommender systems, where generative AI not only enhances recommendation quality but also simplifies the recommendation pipeline.

11 CONCLUSION

In conclusion, our survey is the first to thoroughly examine the retrieval stage of recommender systems, looking at both academic research and industry applications. We cover three main areas: improving similarity computation between users and items using both shallow and deep structures, enhancing indexing mechanisms for efficient retrieval, and optimizing training methods of retrieval. We also conduct extensive benchmarking experiments, testing various methods on three public datasets. Furthermore, we highlight current industrial applications through a case study on retrieval practices at a specific company, detailing the entire process and the challenges they face. This exploration is crucial as the retrieval stage sets the groundwork for the subsequent recommendation processes, determining the efficiency and effectiveness of the entire system.

Additionally, we have identified existing gaps and proposed future directions for research, aiming to inspire and guide ongoing efforts in this field. By discussing both the theoretical frameworks and practical implementations, this survey serves as a valuable resource for researchers and practitioners alike, seeking to enhance or develop innovative retrieval methods. Looking ahead, the field of retrieval in recommender systems holds promising potential for significant advancements. Our hope is that this survey will act as a catalyst for future research, fostering developments that will refine and revolutionize the capabilities of retrieval in recommendation, thus improving the overall user experience on recommendation platforms.

ACKNOWLEDGMENTS

This work is supported by China Merchants Bank Credit Card Center. The Shanghai Jiao Tong University team is partially supported by National Natural Science Foundation of China (62322603, 62177033, 624B2096).

References

- [1] 2003. The Epinions Dataset. <https://alchemy.cs.washington.edu/data/epinions/>.
- [2] 2009. The Flixster Dataset. <https://www.flixster.com/>.
- [3] 2021. The AmazonBooks Dataset. <https://jmcauley.ucsd.edu/data/amazon/amazonbooks>.
- [4] 2021. The Yelp Dataset. <https://www.yelp.com/dataset>.
- [5] 2022. The Criteo Dataset. <https://www.kaggle.com/c/criteo-display-ad-challenge/data>.
- [6] 2022. The Gowalla Dataset. <https://snap.stanford.edu/data/loc-gowalla.html>.
- [7] 2022. The MovieLens Dataset. <https://grouplens.org/datasets/movielens>.
- [8] 2024. The CiteUlike Dataset. <http://www.citeulike.org/>.
- [9] 2024. The MillionSongData Dataset. <http://millionsongdataset.com/>.
- [10] Gediminas Adomavicius and Alexander Tuzhilin. 2005. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE transactions on knowledge and data engineering* 17, 6 (2005), 734–749.
- [11] Gediminas Adomavicius and Alexander Tuzhilin. 2010. Context-aware recommender systems. In *Recommender systems handbook*. Springer, 217–253.
- [12] Ting Bai, Ji-Rong Wen, Jun Zhang, and Wayne Xin Zhao. 2017. A neural collaborative filtering model with interaction-based neighborhood. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 1979–1982.
- [13] Keqin Bao, Jizhi Zhang, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 1007–1014.
- [14] Oren Barkan and Noam Koenigstein. 2016. Item2vec: neural item embedding for collaborative filtering. In *2016 IEEE 26th International Workshop on Machine Learning for Signal Processing (MLSP)*. IEEE, 1–6.
- [15] Zeynep Batmaz, Ali Yurekli, Alper Bilge, and Cihan Kaleli. 2019. A review on deep learning for recommender systems: challenges and remedies. *Artificial Intelligence Review* 52 (2019), 1–37.

- [16] Immanuel Bayer, Xiangnan He, Bhargav Kanagal, and Steffen Rendle. 2017. A generic coordinate descent framework for learning from implicit feedback. In *Proceedings of the 26th international conference on world wide web*. 1341–1350.
- [17] Robert M Bell and Yehuda Koren. 2007. Scalable collaborative filtering with jointly derived neighborhood interpolation weights. In *Seventh IEEE international conference on data mining (ICDM 2007)*. IEEE, 43–52.
- [18] James Bennett and Stan Lanning. 2007. The netflix prize. (2007).
- [19] Rianne van den Berg, Thomas N Kipf, and Max Welling. 2017. Graph convolutional matrix completion. *arXiv preprint arXiv:1706.02263* (2017).
- [20] Jesús Bobadilla, Fernando Ortega, Antonio Hernando, and Abraham Gutiérrez. 2013. Recommender systems survey. *Knowledge-based systems* 46 (2013), 109–132.
- [21] Matthias Böhmer, Brent Hecht, Johannes Schöning, Antonio Krüger, and Gernot Bauer. 2011. Falling asleep with angry birds, facebook and kindle: a large scale study on mobile application usage. In *Proceedings of the 13th international conference on Human computer interaction with mobile devices and services*. 47–56.
- [22] John S Breese, David Heckerman, and Carl Kadie. 2013. Empirical analysis of predictive algorithms for collaborative filtering. *arXiv preprint arXiv:1301.7363* (2013).
- [23] Christopher JC Burges. 2010. From ranknet to lambdarank to lambdamart: An overview. *Learning* 11, 23–581 (2010), 81.
- [24] Da Cao, Xiangnan He, Lianhai Miao, Yuhui An, Chao Yang, and Richang Hong. 2018. Attentive group recommendation. In *The 41st International ACM SIGIR conference on research & development in information retrieval*. 645–654.
- [25] Hugo Caselles-Dupré, Florian Lesaint, and Jimena Royo-Letelier. 2018. Word2vec applied to recommendation: Hyperparameters matter. In *Proceedings of the 12th ACM Conference on Recommender Systems*. 352–356.
- [26] Òscar Celma and Pedro Cano. 2008. From hits to niches? or how popular artists can bias music recommendation and discovery. In *Proceedings of the 2nd KDD workshop on large-scale recommender systems and the netflix prize competition*. 1–8.
- [27] Yukuo Cen, Jianwei Zhang, Xu Zou, Chang Zhou, Hongxia Yang, and Jie Tang. 2020. Controllable multi-interest framework for recommendation. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2942–2951.
- [28] Dong-Kyu Chae, Jin-Soo Kang, Sang-Wook Kim, and Jung-Tae Lee. 2018. CFGAN: A generic collaborative filtering framework based on generative adversarial networks. In *Proceedings of the 27th ACM international conference on information and knowledge management*. 137–146.
- [29] Zheng Chai, Zhihong Chen, Chenliang Li, Rong Xiao, Houyi Li, Jiawei Wu, Jingxu Chen, and Haihong Tang. 2022. User-aware multi-interest learning for candidate matching in recommenders. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1326–1335.
- [30] Chong Chen, Weizhi Ma, Min Zhang, Chenyang Wang, Yiqun Liu, and Shaoping Ma. 2023. Revisiting negative sampling vs. non-sampling in implicit recommendation. *ACM Transactions on Information Systems* 41, 1 (2023), 1–25.
- [31] Chong Chen, Min Zhang, Yongfeng Zhang, Yiqun Liu, and Shaoping Ma. 2020. Efficient neural matrix factorization without sampling for recommendation. *ACM Transactions on Information Systems (TOIS)* 38, 2 (2020), 1–28.
- [32] Gaode Chen, Xinghua Zhang, Yanyan Zhao, Cong Xue, and Ji Xiang. 2021. Exploring periodicity and interactivity in multi-interest framework for sequential recommendation. *arXiv preprint arXiv:2106.04415* (2021).
- [33] Jiawei Chen, Hande Dong, Xiang Wang, Fuli Feng, Meng Wang, and Xiangnan He. 2023. Bias and Debias in Recommender System: A Survey and Future Directions. *ACM Transactions on Information Systems* (Jul 2023), 1–39. <https://doi.org/10.1145/3564284>
- [34] Jin Chen, Defu Lian, Binbin Jin, Xu Huang, Kai Zheng, and Enhong Chen. 2022. Fast variational autoencoder with inverted multi-index for collaborative filtering. In *Proceedings of the ACM Web Conference 2022*. 1944–1954.
- [35] Liang Chen, Yang Liu, Zibin Zheng, and Philip Yu. 2018. Heterogeneous neural attentive factorization machine for rating prediction. In *Proceedings of the 27th ACM international conference on information and knowledge management*. 833–842.
- [36] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior sequence transformer for e-commerce recommendation in alibaba. In *Proceedings of the 1st International Workshop on Deep Learning Practice for High-Dimensional Sparse Data*. 1–4.
- [37] Rui Chen, Qingyi Hua, Yan-Shuo Chang, Bo Wang, Lei Zhang, and Xiangjie Kong. 2018. A survey of collaborative filtering-based recommender systems: From traditional methods to hybrid methods based on social networks. *IEEE access* 6 (2018), 64301–64320.
- [38] Ruey-Cheng Chen, Luke Gallagher, Roi Blanco, and J Shane Culpepper. 2017. Efficient cost-aware cascade ranking in multi-stage retrieval. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 445–454.
- [39] Wanyu Chen, Fei Cai, Honghui Chen, and Maarten De Rijke. 2019. Joint neural collaborative filtering for recommender systems. *ACM Transactions on Information Systems (TOIS)* 37, 4 (2019), 1–30.
- [40] Yoon-Sik Cho and Min-hwan Oh. 2022. Stochastic-expert variational autoencoder for collaborative filtering. In *Proceedings of the ACM Web Conference 2022*. 2482–2490.
- [41] Jaekeol Choi, Euna Jung, Jangwon Suh, and Wonjong Rhee. 2021. Improving bi-encoder document ranking models with two rankers and multi-teacher distillation. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 2192–2196.
- [42] Charles LA Clarke, J Shane Culpepper, and Alistair Moffat. 2016. Assessing efficiency–effectiveness tradeoffs in multi-stage retrieval systems without using relevance judgments. *Information Retrieval Journal* 19 (2016), 351–377.
- [43] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM conference on recommender systems*. 191–198.

- [44] Sunhao Dai, Ninglu Shao, Haiyuan Zhao, Weijie Yu, Zihua Si, Chen Xu, Zhongxiang Sun, Xiao Zhang, and Jun Xu. 2023. Uncovering chatgpt’s capabilities in recommender systems. In Proceedings of the 17th ACM Conference on Recommender Systems. 1126–1132.
- [45] Robin Devooght, Nicolas Kourtellis, and Amin Mantrach. 2015. Dynamic matrix factorization with priors on unknown values. In Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining. 189–198.
- [46] Dario Di Palma, Giovanni Maria Biancofiore, Vito Walter Anelli, Fedelucio Narducci, Tommaso Di Noia, and Eugenio Di Sciascio. 2023. Evaluating chatgpt as a recommender system: A rigorous approach. arXiv preprint arXiv:2309.03613 (2023).
- [47] Jingtao Ding, Yuhuan Quan, Quanming Yao, Yong Li, and Depeng Jin. 2020. Simplify and robustify negative sampling for implicit collaborative filtering. Advances in Neural Information Processing Systems 33 (2020), 1094–1105.
- [48] Ahmed El-Kishky, Thomas Markovich, Kenny Leung, Frank Portman, Aria Haghighi, and Ying Xiao. 2023. k NN-Embed: Locally Smoothed Embedding Mixtures for Multi-interest Candidate Retrieval. In Pacific-Asia Conference on Knowledge Discovery and Data Mining. Springer, 374–386.
- [49] Miao Fan, Jiacheng Guo, Shuai Zhu, Shuo Miao, Mingming Sun, and Ping Li. 2019. MOBIUS: towards the next generation of query-ad matching in baidu’s sponsored search. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2509–2517.
- [50] Hongliang Fei, Jingyuan Zhang, Xingxuan Zhou, Junhao Zhao, Xinyang Qi, and Ping Li. 2021. GemNN: gating-enhanced multi-task neural networks with feature interaction learning for CTR prediction. In Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval. 2166–2171.
- [51] Claudiu S Firan, Wolfgang Nejdl, and Raluca Paiu. 2007. The benefit of using tag-based profiles. In 2007 Latin American Web Conference (LA-WEB 2007). IEEE, 32–41.
- [52] Daniel Fleder and Kartik Hosanagar. 2009. Blockbuster culture’s next rise or fall: The impact of recommender systems on sales diversity. Management science 55, 5 (2009), 697–712.
- [53] Mingsheng Fu, Hong Qu, Zhang Yi, Li Lu, and Yongsheng Liu. 2018. A novel deep learning-based collaborative filtering model for recommendation system. IEEE transactions on cybernetics 49, 3 (2018), 1084–1096.
- [54] Luke Gallagher, Ruey-Cheng Chen, Roi Blanco, and J Shane Culpepper. 2019. Joint optimization of cascade ranking models. In Proceedings of the twelfth ACM international conference on web search and data mining. 15–23.
- [55] Weihao Gao, Xiangjun Fan, Chong Wang, Jiankai Sun, Kai Jia, Wenzhi Xiao, Ruofan Ding, Xingyan Bin, Hui Yang, and Xiaobing Liu. 2020. Deep retrieval: Learning a retrievable structure for large-scale recommendations. arXiv preprint arXiv:2007.07203 (2020).
- [56] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In Proceedings of the 16th ACM Conference on Recommender Systems. 299–315.
- [57] Mihajlo Grbovic and Haibin Cheng. 2018. Real-time personalization using embeddings for search ranking at airbnb. In Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining. 311–320.
- [58] Aditya Grover and Jure Leskovec. 2016. node2vec: Scalable feature learning for networks. In Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining. 855–864.
- [59] Guibing Guo, Huan Zhou, Bowei Chen, Zhirong Liu, Xiao Xu, Xu Chen, Zhenhua Dong, and Xiuqiang He. 2020. IPGAN: Generating informative item pairs by adversarial sampling. IEEE transactions on neural networks and learning systems 33, 2 (2020), 694–706.
- [60] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating large-scale inference with anisotropic vector quantization. In International Conference on Machine Learning. PMLR, 3887–3896.
- [61] Ruining He and Julian McAuley. 2016. VBPR: visual bayesian personalized ranking from implicit feedback. In Proceedings of the AAAI conference on artificial intelligence, Vol. 30.
- [62] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval. 639–648.
- [63] Xiangnan He, Zhankui He, Xiaoyu Du, and Tat-Seng Chua. 2018. Adversarial personalized ranking for recommendation. In The 41st International ACM SIGIR conference on research & development in information retrieval. 355–364.
- [64] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural collaborative filtering. In Proceedings of the 26th international conference on world wide web. 173–182.
- [65] Xiangnan He, Hanwang Zhang, Min-Yen Kan, and Tat-Seng Chua. 2016. Fast matrix factorization for online recommendation with implicit feedback. In Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval. 549–558.
- [66] Zhicheng He, Weiwen Liu, Wei Guo, Jiarui Qin, Yingxue Zhang, Yaochen Hu, and Ruiming Tang. 2023. A survey on user behavior modeling in recommender systems. arXiv preprint arXiv:2302.11087 (2023).
- [67] Jiri Hron, Karl Krauth, Michael Jordan, and Niki Kilbertus. 2021. On component interactions in two-stage recommender systems. Advances in neural information processing systems 34 (2021), 2744–2757.
- [68] Cheng-Kang Hsieh, Longqi Yang, Yin Cui, Tsung-Yi Lin, Serge Belongie, and Deborah Estrin. 2017. Collaborative metric learning. In Proceedings of the 26th international conference on world wide web. 193–201.
- [69] Yifan Hu, Yehuda Koren, and Chris Volinsky. 2008. Collaborative filtering for implicit feedback datasets. In 2008 Eighth IEEE international conference on data mining. Ieee, 263–272.

- [70] Junjie Huang, Guohao Cai, Jieming Zhu, Zhenhua Dong, Ruiming Tang, Weinan Zhang, and Yong Yu. 2024. Recall-Augmented Ranking: Enhancing Click-Through Rate Prediction Accuracy with Cross-Stage Data. In *Companion Proceedings of the ACM on Web Conference 2024*. 830–833.
- [71] Junjie Huang, Jiarui Qin, Jianghao Lin, Ziming Feng, Weinan Zhang, and Yong Yu. 2025. Unleashing the Potential of Multi-Channel Fusion in Retrieval for Personalized Recommendations. In *Proceedings of the ACM on Web Conference 2025*. 483–494.
- [72] Po-Sen Huang, Xiaodong He, Jianfeng Gao, Li Deng, Alex Acero, and Larry Heck. 2013. Learning deep structured semantic models for web search using clickthrough data. In *Proceedings of the 22nd ACM international conference on Information & Knowledge Management*. 2333–2338.
- [73] Tinglin Huang, Yuxiao Dong, Ming Ding, Zhen Yang, Wenzheng Feng, Xinyu Wang, and Jie Tang. 2021. Mixgcf: An improved training method for graph neural network-based recommender systems. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 665–674.
- [74] Xu Huang, Defu Lian, Jin Chen, Liu Zheng, Xing Xie, and Enhong Chen. 2023. Cooperative Retriever and Ranker in Deep Recommenders. In *Proceedings of the ACM Web Conference 2023*. 1150–1161.
- [75] Mohsen Jamali and Martin Ester. 2010. A matrix factorization technique with trust propagation for recommendation in social networks. In *Proceedings of the fourth ACM conference on Recommender systems*. 135–142.
- [76] Binbin Jin, Defu Lian, Zheng Liu, Qi Liu, Jianhui Ma, Xing Xie, and Enhong Chen. 2020. Sampling-decomposable generative adversarial recommender. *Advances in Neural Information Processing Systems* 33 (2020), 22629–22639.
- [77] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [78] Giannis Karamanolakis, Kevin Raji Cherian, Ananth Ravi Narayan, Jie Yuan, Da Tang, and Tony Jebara. 2018. Item recommendation with variational autoencoders and heterogeneous priors. In *Proceedings of the 3rd Workshop on Deep Learning for Recommender Systems*. 10–14.
- [79] George Karypis. 2001. Evaluation of item-based top-n recommendation algorithms. In *Proceedings of the tenth international conference on Information and knowledge management*. 247–254.
- [80] Daeryong Kim and Bongwon Suh. 2019. Enhancing VAEs for collaborative filtering: flexible priors & gating mechanisms. In *Proceedings of the 13th ACM conference on recommender systems*. 403–407.
- [81] Diederik P Kingma and Max Welling. 2013. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114* (2013).
- [82] Yehuda Koren. 2008. Factorization meets the neighborhood: a multifaceted collaborative filtering model. In *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*. 426–434.
- [83] Yehuda Koren. 2009. Collaborative filtering with temporal dynamics. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*. 447–456.
- [84] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.
- [85] John Krumm, Nigel Davies, and Chandra Narayanaswami. 2008. User-generated content. *IEEE Pervasive Computing* 7, 4 (2008), 10–11.
- [86] Wonsung Lee, Kyungwoo Song, and Il-Chul Moon. 2017. Augmented variational autoencoders for collaborative filtering with auxiliary information. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 1139–1148.
- [87] Asher Levi, Osnat Mokryn, Christophe Diot, and Nina Taft. 2012. Finding a needle in a haystack of reviews: cold start context-based hotel recommender system. In *Proceedings of the sixth ACM conference on Recommender systems*. 115–122.
- [88] Chong Li, Kunyang Jia, Dan Shen, C-J Richard Shi, and Hongxia Yang. 2019. Hierarchical Representation Learning for Bipartite Graphs. In *IJCAI*, Vol. 19. 2873–2879.
- [89] Chao Li, Zhiyuan Liu, Mengmeng Wu, Yuchi Xu, Huan Zhao, Pipei Huang, Guoliang Kang, Qiwei Chen, Wei Li, and Dik Lun Lee. 2019. Multi-interest network with dynamic routing for recommendation at Tmall. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 2615–2623.
- [90] Dan Li, Yang Yang, Hongyin Tang, Jingang Wang, Tong Xu, Wei Wu, and Enhong Chen. 2021. VIRT: improving representation-based models for text matching through virtual interaction. *arXiv preprint arXiv:2112.04195* (2021).
- [91] Lei Li, Yongfeng Zhang, and Li Chen. 2023. Prompt distillation for efficient llm-based recommendation. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 1348–1357.
- [92] Xiangyang Li, Bo Chen, HuiFeng Guo, Jingjie Li, Chenxu Zhu, Xiang Long, Sujian Li, Yichao Wang, Wei Guo, Longxia Mao, et al. 2022. Inttower: the next generation of two-tower model for pre-ranking system. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 3292–3301.
- [93] Xiaopeng Li and James She. 2017. Collaborative variational autoencoder for recommender systems. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*. 305–314.
- [94] Xinyi Li, Yongfeng Zhang, and Edward C Malthouse. 2023. Pbnr: Prompt-based news recommender system. *arXiv preprint arXiv:2304.07862* (2023).
- [95] Xinyi Li, Yongfeng Zhang, and Edward C Malthouse. 2023. A preliminary study of chatgpt on news recommendation: Personalization, provider fairness, fake news. *arXiv preprint arXiv:2306.10702* (2023).
- [96] Zekun Li, Zeyu Cui, Shu Wu, Xiaoyu Zhang, and Liang Wang. 2019. Fi-gnn: Modeling feature interactions via graph neural networks for ctr prediction. In *Proceedings of the 28th ACM international conference on information and knowledge management*. 539–548.
- [97] Zhao Li, Xin Shen, Yuhang Jiao, Xuming Pan, Pengcheng Zou, Xianling Meng, Chengwei Yao, and Jiajun Bu. 2020. Hierarchical bipartite graph neural networks: Towards large-scale e-commerce applications. In *2020 IEEE 36th International Conference on Data Engineering (ICDE)*. IEEE, 1677–1688.

- [98] Dawen Liang, Laurent Charlin, James McInerney, and David M Blei. 2016. Modeling user exposure in recommendation. In Proceedings of the 25th international conference on World Wide Web. 951–961.
- [99] Dawen Liang, Rahul G Krishnan, Matthew D Hoffman, and Tony Jebara. 2018. Variational autoencoders for collaborative filtering. In Proceedings of the 2018 world wide web conference. 689–698.
- [100] Huizhi Liang, Yue Xu, Yuefeng Li, Richi Nayak, and Xiaohui Tao. 2010. Connecting users and items with weighted tags for personalized item recommendations. In Proceedings of the 21st ACM conference on Hypertext and hypermedia. 51–60.
- [101] Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Hao Zhang, Yong Liu, Chuhan Wu, Xiangyang Li, Chenxu Zhu, et al. 2023. How can recommender systems benefit from large language models: A survey. arXiv preprint arXiv:2306.05817 (2023).
- [102] Jianghao Lin, Yanru Qu, Wei Guo, Xinyi Dai, Ruiming Tang, Yong Yu, and Weinan Zhang. 2023. Map: A model-agnostic pretraining framework for click-through rate prediction. In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 1384–1395.
- [103] Jianghao Lin, Rong Shan, Chenxu Zhu, Kounianhua Du, Bo Chen, Shigang Quan, Ruiming Tang, Yong Yu, and Weinan Zhang. 2024. Rella: Retrieval-enhanced large language models for lifelong sequential behavior comprehension in recommendation. In Proceedings of the ACM on Web Conference 2024. 3497–3508.
- [104] Greg Linden, Brent Smith, and Jeremy York. 2003. Amazon.com recommendations: Item-to-item collaborative filtering. IEEE Internet computing 7, 1 (2003), 76–80.
- [105] Junling Liu, Chao Liu, Peilin Zhou, Renjie Lv, Kang Zhou, and Yan Zhang. 2023. Is chatgpt a good recommender? a preliminary study. arXiv preprint arXiv:2304.10149 (2023).
- [106] Jie Liu, Dong Wang, and Yue Ding. 2017. PHD: A probabilistic model of hybrid deep collaborative filtering for recommender systems. In Asian Conference on machine learning. PMLR, 224–239.
- [107] Shang Liu, Zhenzhong Chen, Hongyi Liu, and Xinghai Hu. 2019. User-video co-attention network for personalized micro-video recommendation. In The World Wide Web Conference. 3020–3026.
- [108] Zheng Liu, Jianxun Lian, Junhan Yang, Defu Lian, and Xing Xie. 2020. Octopus: Comprehensive and elastic user representation for the generation of recommendation candidates. In Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval. 289–298.
- [109] Zhiwei Liu, Lin Meng, Fei Jiang, Jiawei Zhang, and Philip S Yu. 2020. Deoscillated graph collaborative filtering. arXiv preprint arXiv:2011.02100 (2020).
- [110] Sam Lobel, Chunyuan Li, Jianfeng Gao, and Lawrence Carin. 2019. Towards amortized ranking-critical training for collaborative filtering. arXiv preprint arXiv:1906.04281 (2019).
- [111] Yuxiang Lu, Yiding Liu, Jiaxiang Liu, Yunsheng Shi, Zhengjie Huang, Shikun Feng Yu Sun, Hao Tian, Hua Wu, Shuaiqiang Wang, Dawei Yin, et al. 2022. Ernie-search: Bridging cross-encoder with dual-encoder via self on-the-fly distillation for dense passage retrieval. arXiv preprint arXiv:2205.09153 (2022).
- [112] Jianxin Ma, Chang Zhou, Peng Cui, Hongxia Yang, and Wenwu Zhu. 2019. Learning disentangled representations for recommendation. Advances in neural information processing systems 32 (2019).
- [113] Jianxin Ma, Chang Zhou, Hongxia Yang, Peng Cui, Xin Wang, and Wenwu Zhu. 2020. Disentangled self-supervision in sequential recommenders. In Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 483–491.
- [114] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE transactions on pattern analysis and machine intelligence 42, 4 (2018), 824–836.
- [115] Jarana Manotumruksa, Craig Macdonald, and Iadh Ounis. 2017. A deep recurrent collaborative filtering framework for venue recommendation. In Proceedings of the 2017 ACM on Conference on Information and Knowledge Management. 1429–1438.
- [116] Kelong Mao, Jieming Zhu, Jinpeng Wang, Quanyu Dai, Zhenhua Dong, Xi Xiao, and Xiuqiang He. 2021. SimpleX: A simple and strong baseline for collaborative filtering. In Proceedings of the 30th ACM International Conference on Information & Knowledge Management. 1243–1252.
- [117] Kelong Mao, Jieming Zhu, Xi Xiao, Biao Lu, Zhaowei Wang, and Xiuqiang He. 2021. UltraGCN: ultra simplification of graph convolutional networks for recommendation. In Proceedings of the 30th ACM international conference on information & knowledge management. 1253–1262.
- [118] Prem Melville, Raymond J Mooney, Ramadass Nagarajan, et al. 2002. Content-boosted collaborative filtering for improved recommendations. Aaai/iaai 23 (2002), 187–192.
- [119] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. Advances in neural information processing systems 26 (2013).
- [120] Xia Ning and George Karypis. 2011. Slim: Sparse linear methods for top-n recommender systems. In 2011 IEEE 11th international conference on data mining. IEEE, 497–506.
- [121] Chihiro Ono, Yasuhiro Takishima, Yoichi Motomura, and Hideki Asoh. 2009. Context-aware preference model based on a study of difference between real and supposed situation data. In User Modeling, Adaptation, and Personalization: 17th International Conference, UMAP 2009, formerly UM and AH, Trento, Italy, June 22-26, 2009. Proceedings 17. Springer, 102–113.
- [122] Razaq Otunba, Raimi A Rufai, and Jessica Lin. 2019. Deep stacked ensemble recommender. In Proceedings of the 31st International Conference on Scientific and Statistical Database Management. 197–201.
- [123] Aditya Pal, Chantat Eksombatchai, Yitong Zhou, Bo Zhao, Charles Rosenberg, and Jure Leskovec. 2020. Pinnersage: Multi-modal user embedding framework for recommendations at pinterest. In Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery &

- Data Mining. 2311–2320.
- [124] Haolei Pei, Yuanyuan Xu, Yangping Zhu, and Yuan Nie. 2024. HimiRec: Modeling Hierarchical Multi-interest for Recommendation. arXiv preprint arXiv:2402.01253 (2024).
 - [125] Bryan Perozzi, Rami Al-Rfou, and Steven Skiena. 2014. Deepwalk: Online learning of social representations. In Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining. 701–710.
 - [126] Qi Pi, Weijie Bian, Guorui Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Practice on long sequential user behavior modeling for click-through rate prediction. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. 2671–2679.
 - [127] Qi Pi, Guorui Zhou, Yujing Zhang, Zhe Wang, Lejian Ren, Ying Fan, Xiaoqiang Zhu, and Kun Gai. 2020. Search-based user interest modeling with lifelong sequential behavior data for click-through rate prediction. In Proceedings of the 29th ACM International Conference on Information & Knowledge Management. 2685–2692.
 - [128] Jiarui Qin, Weinan Zhang, Rong Su, Zhirong Liu, Weiwen Liu, Ruiming Tang, Xiuqiang He, and Yong Yu. 2021. Retrieval & interaction machine for tabular data prediction. In Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining. 1379–1389.
 - [129] Jiarui Qin, Jiachen Zhu, Bo Chen, Zhirong Liu, Weiwen Liu, Ruiming Tang, Rui Zhang, Yong Yu, and Weinan Zhang. 2022. RankFlow: Joint Optimization of Multi-Stage Cascade Ranking Systems as Flows. In Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval. 814–824.
 - [130] Yanru Qu, Han Cai, Kan Ren, Weinan Zhang, Yong Yu, Ying Wen, and Jun Wang. 2016. Product-based neural networks for user response prediction. In 2016 IEEE 16th international conference on data mining (ICDM). IEEE, 1149–1154.
 - [131] Steffen Rendle. 2010. Factorization machines. In 2010 IEEE International conference on data mining. IEEE, 995–1000.
 - [132] Steffen Rendle and Christoph Freudenthaler. 2014. Improving pairwise learning for item recommendation from implicit feedback. In Proceedings of the 7th ACM international conference on Web search and data mining. 273–282.
 - [133] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2012. BPR: Bayesian personalized ranking from implicit feedback. arXiv preprint arXiv:1205.2618 (2012).
 - [134] Stephen E Robertson, Steve Walker, Susan Jones, Micheline M Hancock-Beaulieu, Mike Gattford, et al. 1995. Okapi at TREC-3. Nist Special Publication Sp 109 (1995), 109.
 - [135] Valentin Robu, Harry Halpin, and Hana Shepherd. 2009. Emergence of consensus and shared vocabularies in collaborative tagging systems. ACM Transactions on the Web (TWEB) 3, 4 (2009), 1–34.
 - [136] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2000. Analysis of recommendation algorithms for e-commerce. In Proceedings of the 2nd ACM Conference on Electronic Commerce. 158–167.
 - [137] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In Proceedings of the 10th international conference on World Wide Web. 285–295.
 - [138] Ilya Shenbin, Anton Alekseev, Elena Tutubalina, Valentin Malykh, and Sergey I Nikolenko. 2020. Recvae: A new variational autoencoder for top-n recommendations with implicit feedback. In Proceedings of the 13th international conference on web search and data mining. 528–536.
 - [139] Andriy Shepitsen, Jonathan Gemmell, Bamshad Mobasher, and Robin Burke. 2008. Personalized recommendation in social tagging systems using hierarchical clustering. In Proceedings of the 2008 ACM conference on Recommender systems. 259–266.
 - [140] Hui Shi, Yupeng Gu, Yitong Zhou, Bo Zhao, Sicun Gao, and Jishen Zhao. 2023. Everyone’s preference changes differently: A weighted multi-interest model for retrieval. In International Conference on Machine Learning. PMLR, 31228–31242.
 - [141] Yue Shi, Martha Larson, and Alan Hanjalic. 2014. Collaborative filtering beyond the user-item matrix: A survey of the state of the art and future challenges. ACM Computing Surveys (CSUR) 47, 1 (2014), 1–45.
 - [142] Xiaoyuan Su and Taghi M Khoshgoftaar. 2009. A survey of collaborative filtering techniques. Advances in artificial intelligence 2009 (2009).
 - [143] Jianing Sun, Yingxue Zhang, Wei Guo, Huifeng Guo, Ruiming Tang, Xiuqiang He, Chen Ma, and Mark Coates. 2020. Neighbor interaction aware graph convolution networks for recommendation. In Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval. 1289–1298.
 - [144] Jianing Sun, Yingxue Zhang, Chen Ma, Mark Coates, Huifeng Guo, Ruiming Tang, and Xiuqiang He. 2019. Multi-graph convolution collaborative filtering. In 2019 IEEE international conference on data mining (ICDM). IEEE, 1306–1311.
 - [145] James Surowiecki. 2005. The wisdom of crowds. Anchor.
 - [146] Gábor Takács, István Pilászy, Bottyán Németh, and Domonkos Tikk. 2007. Major components of the gravity recommendation system. Acm Sigkdd Explorations Newsletter 9, 2 (2007), 80–83.
 - [147] Gábor Takács, István Pilászy, Bottyán Németh, and Domonkos Tikk. 2008. Matrix factorization and neighbor based algorithms for the netflix prize problem. In Proceedings of the 2008 ACM conference on Recommender systems. 267–274.
 - [148] Qiaoyu Tan, Ninghao Liu, Xing Zhao, Hongxia Yang, Jingren Zhou, and Xia Hu. 2020. Learning to hash with graph neural networks for recommender systems. In Proceedings of The Web Conference 2020. 1988–1998.
 - [149] Qiaoyu Tan, Jianwei Zhang, Jiangchao Yao, Ninghao Liu, Jingren Zhou, Hongxia Yang, and Xia Hu. 2021. Sparse-interest network for sequential recommendation. In Proceedings of the 14th ACM international conference on web search and data mining. 598–606.
 - [150] Jiakai Tang, Sunhao Dai, Zexu Sun, Xu Chen, Jun Xu, Wenhui Yu, Lantao Hu, Peng Jiang, and Han Li. 2024. Towards Robust Recommendation via Decision Boundary-aware Graph Contrastive Learning. In Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 2854–2865.

- [151] Jian Tang, Meng Qu, Mingzhe Wang, Ming Zhang, Jun Yan, and Qiaozhu Mei. 2015. Line: Large-scale information network embedding. In *Proceedings of the 24th international conference on world wide web*. 1067–1077.
- [152] Jakub Tomczak and Max Welling. 2018. VAE with a VampPrior. In *International conference on artificial intelligence and statistics*. PMLR, 1214–1223.
- [153] Quoc-Tuan Truong, Aghiles Salah, and Hady W Lauw. 2021. Bilateral variational autoencoder for collaborative filtering. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*. 292–300.
- [154] Karen HL Tso-Sutter, Leandro Balby Marinho, and Lars Schmidt-Thieme. 2008. Tag-aware recommender systems by fusion of collaborative filtering algorithms. In *Proceedings of the 2008 ACM symposium on Applied computing*. 1995–1999.
- [155] Daixin Wang, Peng Cui, and Wenwu Zhu. 2016. Structural deep network embedding. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*. 1225–1234.
- [156] Hangyu Wang, Jianghao Lin, Xiangyang Li, Bo Chen, Chenxu Zhu, Ruiming Tang, Weinan Zhang, and Yong Yu. 2023. FLIP: Towards Fine-grained Alignment between ID-based Models and Pretrained Language Models for CTR Prediction. *arXiv e-prints* (2023), arXiv–2310.
- [157] Haoyu Wang, Nan Shao, and Defu Lian. 2019. Adversarial binary collaborative filtering for implicit feedback. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 5248–5255.
- [158] Hongwei Wang, Fuzheng Zhang, Xing Xie, and Minyi Guo. 2018. DKN: Deep knowledge-aware network for news recommendation. In *Proceedings of the 2018 world wide web conference*. 1835–1844.
- [159] Jizhe Wang, Pipei Huang, Huan Zhao, Zhibo Zhang, Binqiang Zhao, and Dik Lun Lee. 2018. Billion-scale commodity embedding for e-commerce recommendation in alibaba. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 839–848.
- [160] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, et al. 2021. Milvus: A purpose-built vector data management system. In *Proceedings of the 2021 International Conference on Management of Data*. 2614–2627.
- [161] Jun Wang, Lantao Yu, Weinan Zhang, Yu Gong, Yinghui Xu, Benyou Wang, Peng Zhang, and Dell Zhang. 2017. Irgan: A minimax game for unifying generative and discriminative information retrieval models. In *Proceedings of the 40th International ACM SIGIR conference on Research and Development in Information Retrieval*. 515–524.
- [162] Lidan Wang, Jimmy Lin, and Donald Metzler. 2011. A cascade ranking model for efficient ranked retrieval. In *Proceedings of the 34th international ACM SIGIR conference on Research and development in Information Retrieval*. 105–114.
- [163] Siyu Wang, Xiaocong Chen, Quan Z Sheng, Yihong Zhang, and Lina Yao. 2023. Causal disentangled variational auto-encoder for preference understanding in recommendation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1874–1878.
- [164] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. In *Proceedings of the 42nd international ACM SIGIR conference on Research and development in Information Retrieval*. 165–174.
- [165] Xiang Wang, Hongye Jin, An Zhang, Xiangnan He, Tong Xu, and Tat-Seng Chua. 2020. Disentangled graph collaborative filtering. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*. 1001–1010.
- [166] Xiao Wang, Ruijia Wang, Chuan Shi, Guojie Song, and Qingyong Li. 2020. Multi-component graph convolutional collaborative filtering. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 6267–6274.
- [167] Xiang Wang, Yaokun Xu, Xiangnan He, Yixin Cao, Meng Wang, and Tat-Seng Chua. 2020. Reinforced negative sampling over knowledge graph for recommendation. In *Proceedings of the web conference 2020*. 99–109.
- [168] Yanmeng Wang, Jun Bai, Ye Wang, Jianfei Zhang, Wenge Rong, Zongcheng Ji, Shaojun Wang, and Jing Xiao. 2022. Enhancing dual-encoders with question and answer cross-embeddings for answer retrieval. *arXiv preprint arXiv:2206.02978* (2022).
- [169] Yancheng Wang, Ziyang Jiang, Zheng Chen, Fan Yang, Yingxue Zhou, Eunah Cho, Xing Fan, Xiaojiang Huang, Yanbin Lu, and Yingzhen Yang. 2023. Recmind: Large language model powered agent for recommendation. *arXiv preprint arXiv:2308.14296* (2023).
- [170] Yunli Wang, Zhiqiang Wang, Jian Yang, Shiyang Wen, Dongying Kong, Han Li, and Kun Gai. 2023. Adaptive Neural Ranking Framework: Toward Maximized Business Goal for Cascade Ranking Systems. *arXiv preprint arXiv:2310.10462* (2023).
- [171] Zhe Wang, Liqin Zhao, Biye Jiang, Guorui Zhou, Xiaoqiang Zhu, and Kun Gai. 2020. Cold: Towards the next generation of pre-ranking system. *arXiv preprint arXiv:2007.16122* (2020).
- [172] Jiancan Wu, Xiang Wang, Fuli Feng, Xiangnan He, Liang Chen, Jianxun Lian, and Xing Xie. 2021. Self-supervised graph learning for recommendation. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 726–735.
- [173] Le Wu, Xiangnan He, Xiang Wang, Kun Zhang, and Meng Wang. 2022. A survey on accuracy-oriented neural recommendation: From collaborative filtering to information-rich recommendation. *IEEE Transactions on Knowledge and Data Engineering* 35, 5 (2022), 4425–4445.
- [174] Yao Wu, Christopher DuBois, Alice X Zheng, and Martin Ester. 2016. Collaborative denoising auto-encoders for top-n recommender systems. In *Proceedings of the ninth ACM international conference on web search and data mining*. 153–162.
- [175] Yunjia Xi, Jianghao Lin, Weiwen Liu, Xinyi Dai, Weinan Zhang, Rui Zhang, Ruiming Tang, and Yong Yu. 2023. A bird’s-eye view of reranking: from list level to page level. In *Proceedings of the Sixteenth ACM International Conference on Web Search and Data Mining*. 1075–1083.
- [176] Yunjia Xi, Weiwen Liu, Jianghao Lin, Jieming Zhu, Bo Chen, Ruiming Tang, Weinan Zhang, Rui Zhang, and Yong Yu. 2023. Towards open-world recommendation with knowledge augmentation from large language models. *arXiv preprint arXiv:2306.10933* (2023).
- [177] Jun Xiao, Hao Ye, Xiangnan He, Hanwang Zhang, Fei Wu, and Tat-Seng Chua. 2017. Attentional factorization machines: Learning the weight of feature interactions via attention networks. *arXiv preprint arXiv:1708.04617* (2017).

- [178] Yueqi Xie, Jingqi Gao, Peilin Zhou, Qichen Ye, Yining Hua, Jae Boum Kim, Fangzhao Wu, and Sunghun Kim. 2023. Rethinking multi-interest learning for candidate matching in recommender systems. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 283–293.
- [179] Xin Xin, Fajie Yuan, Xiangnan He, and Joemon M Jose. 2018. Batch is not heavy: Learning word representations from all samples. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 1853–1862.
- [180] Liang Xiong, Xi Chen, Tzu-Kuo Huang, Jeff Schneider, and Jaime G Carbonell. 2010. Temporal collaborative filtering with bayesian probabilistic tensor factorization. In *Proceedings of the 2010 SIAM international conference on data mining*. SIAM, 211–222.
- [181] Tianyu Xiong and Xiaohan Yu. 2024. Multi-Tower Multi-Interest Recommendation with User Representation Repel. *arXiv preprint arXiv:2403.05122* (2024).
- [182] Chen Xu, Quan Li, Junfeng Ge, Jinyang Gao, Xiaoyong Yang, Changhua Pei, Fei Sun, Jian Wu, Hanxiao Sun, and Wenwu Ou. 2020. Privileged features distillation at taobao recommendations. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2590–2598.
- [183] Jun Xu, Xiangnan He, and Hang Li. 2018. Deep learning for matching in search and recommendation. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*. 1365–1368.
- [184] Shuyuan Xu, Wenyue Hua, and Yongfeng Zhang. 2023. Openp5: Benchmarking foundation models for recommendation. *arXiv preprint arXiv:2306.11134* (2023).
- [185] Zhenhui Xu, Meng Zhao, Lique Liu, Lei Xiao, Xiaopeng Zhang, and Bifeng Zhang. 2022. Mixture of virtual-kernel experts for multi-objective user profile modeling. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4257–4267.
- [186] Feng Xue, Xiangnan He, Xiang Wang, Jiandong Xu, Kai Liu, and Richang Hong. 2019. Deep item-based collaborative filtering for top-n recommendation. *ACM Transactions on Information Systems (TOIS)* 37, 3 (2019), 1–25.
- [187] Ji Yang, Xinyang Yi, Derek Zhiyuan Cheng, Lichan Hong, Yang Li, Simon Xiaoming Wang, Taibai Xu, and Ed H Chi. 2020. Mixed negative sampling for learning two-tower neural networks in recommendations. In *Companion proceedings of the web conference 2020*. 441–447.
- [188] Xiaoyong Yang, Yadong Zhu, Yi Zhang, Xiaobo Wang, and Quan Yuan. 2020. Large scale product graph construction for recommendation in e-commerce. *arXiv preprint arXiv:2010.05525* (2020).
- [189] Haibo Ye, Xinjie Li, Yuan Yao, and Hanghang Tong. 2023. Towards robust neural graph collaborative filtering via structure denoising and embedding perturbation. *ACM Transactions on Information Systems* 41, 3 (2023), 1–28.
- [190] Baolin Yi, Xiaoxuan Shen, Hai Liu, Zhaoli Zhang, Wei Zhang, Sannyuya Liu, and Naixue Xiong. 2019. Deep matrix factorization with implicit feedback embedding for recommendation system. *IEEE Transactions on Industrial Informatics* 15, 8 (2019), 4591–4601.
- [191] Xinyang Yi, Ji Yang, Lichan Hong, Derek Zhiyuan Cheng, Lukasz Heldt, Aditee Kumthekar, Zhe Zhao, Li Wei, and Ed Chi. 2019. Sampling-bias-corrected neural modeling for large corpus item recommendations. In *Proceedings of the 13th ACM Conference on Recommender Systems*. 269–277.
- [192] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 974–983.
- [193] Hsiang-Fu Yu, Mikhail Bilenko, and Chih-Jen Lin. 2017. Selection of negative samples for one-class matrix factorization. In *Proceedings of the 2017 SIAM International Conference on Data Mining*. SIAM, 363–371.
- [194] Junliang Yu, Hongzhi Yin, Xin Xia, Tong Chen, Lizhen Cui, and Quoc Viet Hung Nguyen. 2022. Are graph augmentations necessary? simple graph contrastive learning for recommendation. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*. 1294–1303.
- [195] Wenhui Yu and Zheng Qin. 2020. Sampler design for implicit feedback data by noisy-label robust learning. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*. 861–870.
- [196] Yantao Yu, Weipeng Wang, Zhoutian Feng, and Daiyue Xue. 2021. A dual augmented two-tower model for online large-scale recommendation. *DLP-KDD* (2021).
- [197] Jizhi Zhang, Keqin Bao, Yang Zhang, Wenjie Wang, Fuli Feng, and Xiangnan He. 2023. Is chatgpt fair for recommendation? evaluating fairness in large language model recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*. 993–999.
- [198] Junjie Zhang, Ruobing Xie, Yupeng Hou, Wayne Xin Zhao, Leyu Lin, and Ji-Rong Wen. 2023. Recommendation as instruction following: A large language model empowered recommendation approach. *arXiv preprint arXiv:2305.07001* (2023).
- [199] Shengyu Zhang, Lingxiao Yang, Dong Yao, Yujie Lu, Fuli Feng, Zhou Zhao, Tat-Seng Chua, and Fei Wu. 2022. Re4: Learning to re-contrast, re-attend, re-construct for multi-interest recommendation. In *Proceedings of the ACM Web Conference 2022*. 2216–2226.
- [200] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. 2019. Deep learning based recommender system: A survey and new perspectives. *ACM computing surveys (CSUR)* 52, 1 (2019), 1–38.
- [201] Weinan Zhang, Tianqi Chen, Jun Wang, and Yong Yu. 2013. Optimizing top-n collaborative filtering via dynamic negative item sampling. In *Proceedings of the 36th international ACM SIGIR conference on Research and development in information retrieval*. 785–788.
- [202] Weinan Zhang, Tianming Du, and Jun Wang. 2016. Deep Learning over Multi-field Categorical Data: –A Case Study on User Response Prediction. In *Advances in Information Retrieval: 38th European Conference on IR Research, ECIR 2016, Padua, Italy, March 20–23, 2016. Proceedings 38*. Springer, 45–57.

- [203] Weinan Zhang, Jiarui Qin, Wei Guo, Ruiming Tang, and Xiuqiang He. 2021. Deep learning for click-through rate estimation. arXiv preprint arXiv:2104.10584 (2021).
- [204] Xiliang Zhang, Jin Liu, Siwei Chang, Peizhu Gong, Zhongdai Wu, and Bing Han. 2023. MIRN: A multi-interest retrieval network with sequence-to-interest EM routing. Plos one 18, 2 (2023), e0281275.
- [205] Wayne Xin Zhao, Shanlei Mu, Yupeng Hou, Zihan Lin, Yushuo Chen, Xingyu Pan, Kaiyuan Li, Yujie Lu, Hui Wang, Changxin Tian, et al. 2021. Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. In proceedings of the 30th acm international conference on information & knowledge management. 4653–4664.
- [206] Kai Zheng, Haijun Zhao, Rui Huang, Beichuan Zhang, Na Mou, Yanan Niu, Yang Song, Hongning Wang, and Kun Gai. 2024. Full Stage Learning to Rank: A Unified Framework for Multi-Stage Systems. arXiv preprint arXiv:2405.04844 (2024).
- [207] Lei Zheng, Chun-Ta Lu, Fei Jiang, Jiawei Zhang, and Philip S Yu. 2018. Spectral collaborative filtering. In Proceedings of the 12th ACM conference on recommender systems. 311–319.
- [208] Aakas Zhiyuli, Yanfang Chen, Xuan Zhang, and Xun Liang. 2023. Bookgpt: A general framework for book recommendation empowered by large language model. arXiv preprint arXiv:2305.15673 (2023).
- [209] Chang Zhou, Jianxin Ma, Jianwei Zhang, Jingren Zhou, and Hongxia Yang. 2021. Contrastive learning for debiased candidate generation in large-scale recommender systems. In Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining. 3985–3995.
- [210] Jianing Zhou, Junhao Wen, Shun Li, and Wei Zhou. 2019. From content text encoding perspective: A hybrid deep matrix factorization approach for recommender system. In 2019 International Joint Conference on Neural Networks (IJCNN). IEEE, 1–8.
- [211] Yao Zhou, Jianpeng Xu, Jun Wu, Zeinab Taghavi, Evren Korpoglu, Kannan Achan, and Jingrui He. 2021. Pure: Positive-unlabeled recommendation with generative adversarial network. In Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining. 2409–2419.
- [212] Han Zhu, Daqing Chang, Ziru Xu, Pengye Zhang, Xiang Li, Jie He, Han Li, Jian Xu, and Kun Gai. 2019. Joint optimization of tree-based index and deep model for recommender systems. Advances in Neural Information Processing Systems 32 (2019).
- [213] Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning tree-based deep model for recommender systems. In Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining. 1079–1088.
- [214] Jieming Zhu, Guohao Cai, Junjie Huang, Zhenhua Dong, Ruiming Tang, and Weinan Zhang. 2023. ReLoop2: Building Self-Adaptive Recommendation Models via Responsive Error Compensation Loop. In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. 5728–5738.
- [215] Jieming Zhu, Quanyu Dai, Liangcai Su, Rong Ma, Jinyang Liu, Guohao Cai, Xi Xiao, and Rui Zhang. 2022. Bars: Towards open benchmarking for recommender systems. In Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval. 2912–2923.
- [216] Yaochen Zhu and Zhenzhong Chen. 2022. Mutually-regularized dual collaborative variational auto-encoder for recommendation systems. In Proceedings of The ACM Web Conference 2022. 2379–2387.
- [217] Jingwei Zhuo, Ziru Xu, Wei Dai, Han Zhu, Han Li, Jian Xu, and Kun Gai. 2020. Learning optimal tree models under beam search. In International Conference on Machine Learning. PMLR, 11650–11659.